

1

Visual Studio C# a .NET

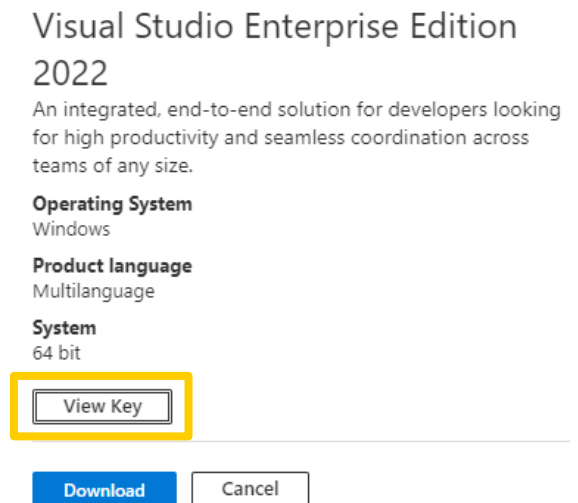
Inštalácia a základy

Visual Studio

Inštalácia a aktivácia

Visual Studio – odkazy na stiahnutie

- Najnovšie **Visual Studio** si môžete stiahnuť z:
 - <https://www.visualstudio.com/downloads/>
 - <https://azureforeducation.microsoft.com/devtools> ([Azure Dev Tools For Teaching](#)), kde je okrem Visual Studia k dispozícii aj iný softvér od Microsoftu zadarmo – viac informácií a návod: <https://ikt.uniza.sk/uniza-wiki/azure-dev-tools-for-teaching/>
- Stačiť by Vám mala verzia **Community**, ktorá je zadarmo nielen na akademické, ale aj komerčné použitie (za určitých podmienok)
- V učebniach máme nainštalovanú najvyššiu verziu **Enterprise** – ak ju chcete používať aj doma, nezabudnite získať kľúč cez **View Key** v Azure Dev Tools For Teaching a potom ho aplikovať vo Visual Studio



Visual Studio – inštalácia cez WinGet na Windows

- Ak ešte nemáte nainštalované najnovšie Visual Studio 2022 vo verzii **Enterprise** alebo **Community**, môžete využiť aj WinGet príkaz na Windows 10/11 nasledovne – stačí spustiť cez cmd / Terminal:

```
winget install --id Microsoft.VisualStudio.2022.Enterprise --override "--add  
Microsoft.VisualStudio.Workload.CoreEditor;includeRecommended --add  
Microsoft.VisualStudio.Workload.NetWeb;includeRecommended --add  
Microsoft.VisualStudio.Workload.NetCrossPlat;includeRecommended --add  
Microsoft.VisualStudio.Workload.ManagedDesktop;includeRecommended --add  
Microsoft.VisualStudio.Workload.Universal;includeRecommended --add  
Microsoft.VisualStudio.Workload.Data;includeRecommended --add  
Microsoft.VisualStudio.Component.ClassDesigner"
```

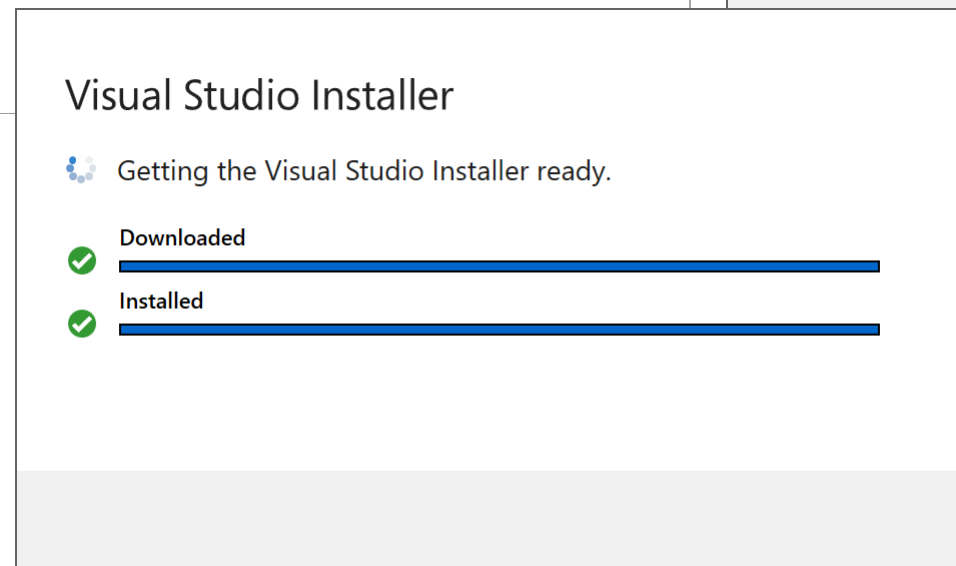
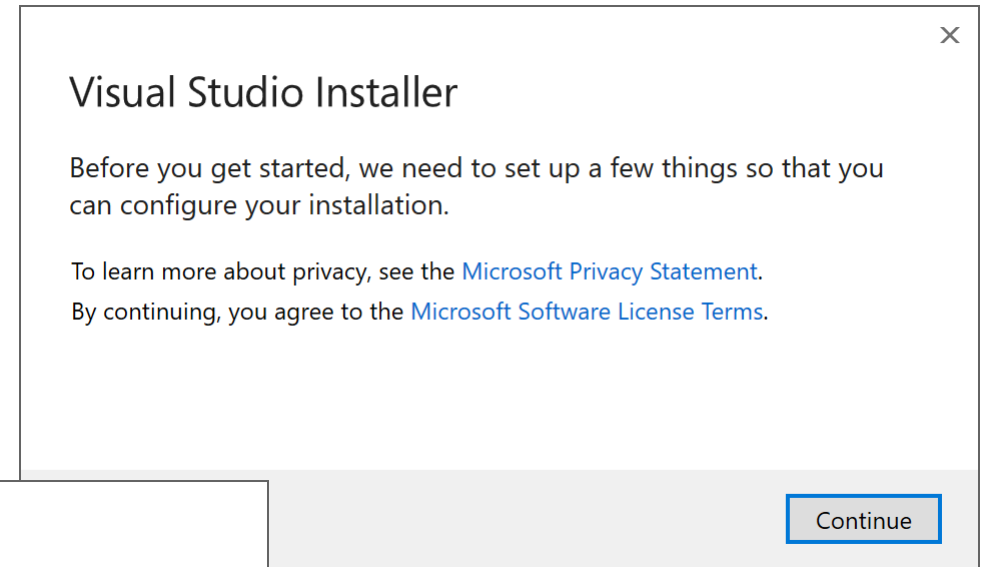
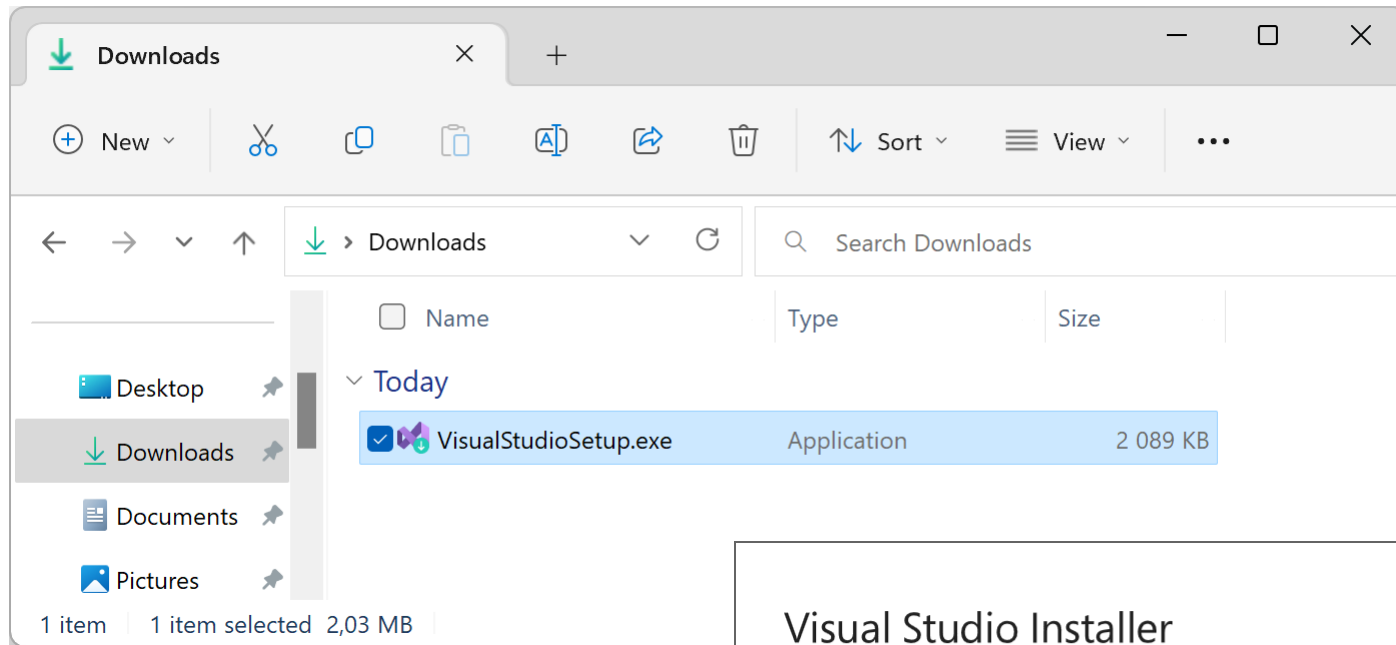
- Zoznam inštalovaných workloadov a komponentov:
 - Enterprise: <https://learn.microsoft.com/en-us/visualstudio/install/workload-component-id-vs-enterprise?view=vs-2022>
 - Community: <https://learn.microsoft.com/en-us/visualstudio/install/workload-component-id-vs-community?view=vs-2022>

Visual Studio – porovnanie verzií

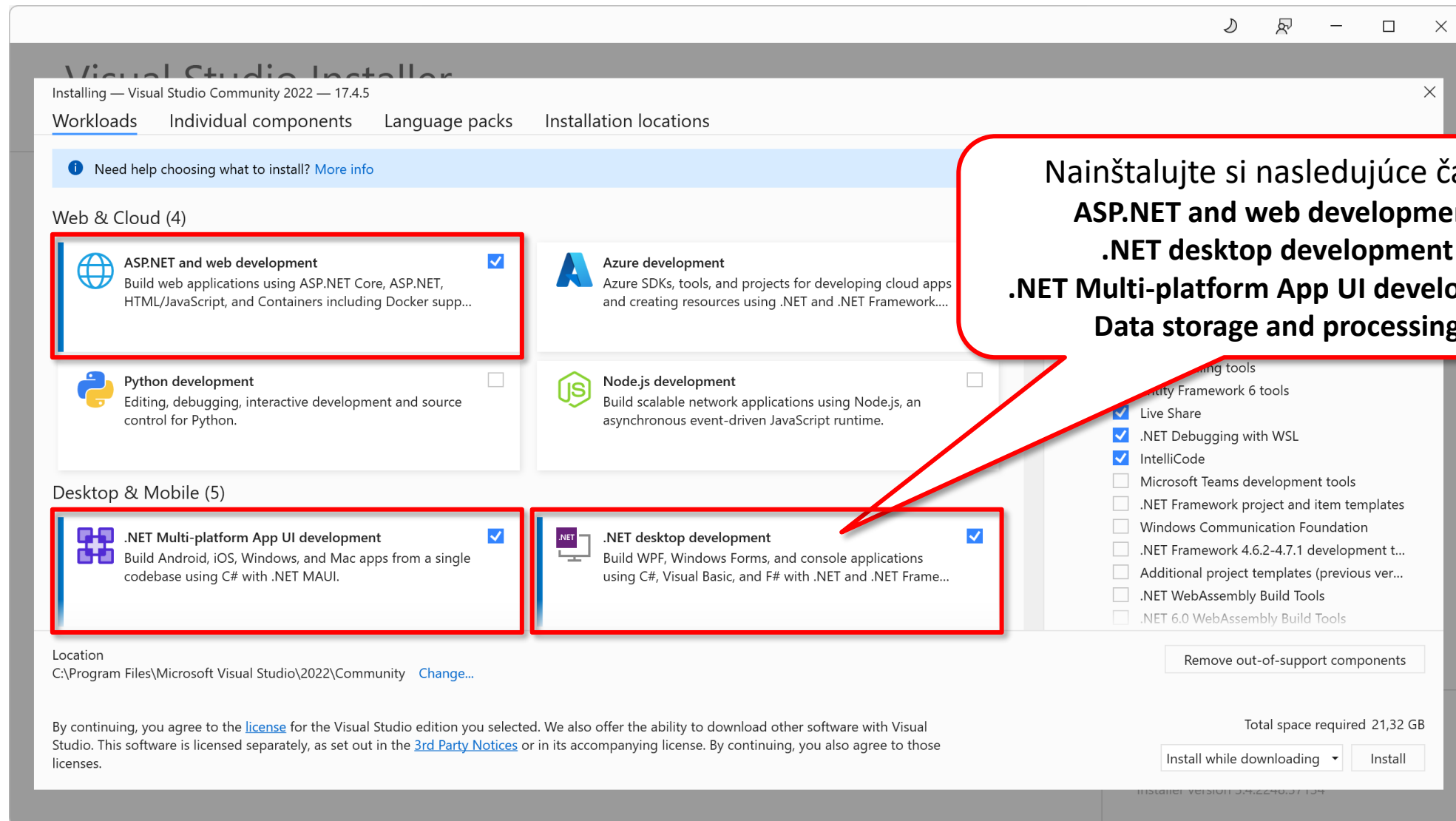
Supported Features	Visual Studio Community Free download	Visual Studio Professional Buy	Visual Studio Enterprise Buy
⊕ Supported Usage Scenarios	● ● ● ○	● ● ● ●	● ● ● ●
Development Platform Support ²	● ● ● ●	● ● ● ●	● ● ● ●
⊕ Integrated Development Environment	● ● ● ○	● ● ● ○	● ● ● ●
⊕ Advanced Debugging and Diagnostics	● ● ○ ○	● ● ○ ○	● ● ● ●
⊕ Testing Tools	● ○ ○ ○	● ○ ○ ○	● ● ● ●
⊕ Cross-platform Development	● ● ○ ○	● ● ○ ○	● ● ● ●
⊕ Collaboration Tools and Features	● ● ● ●	● ● ● ●	● ● ● ●

Zdroj: <https://visualstudio.microsoft.com/vs/compare/>

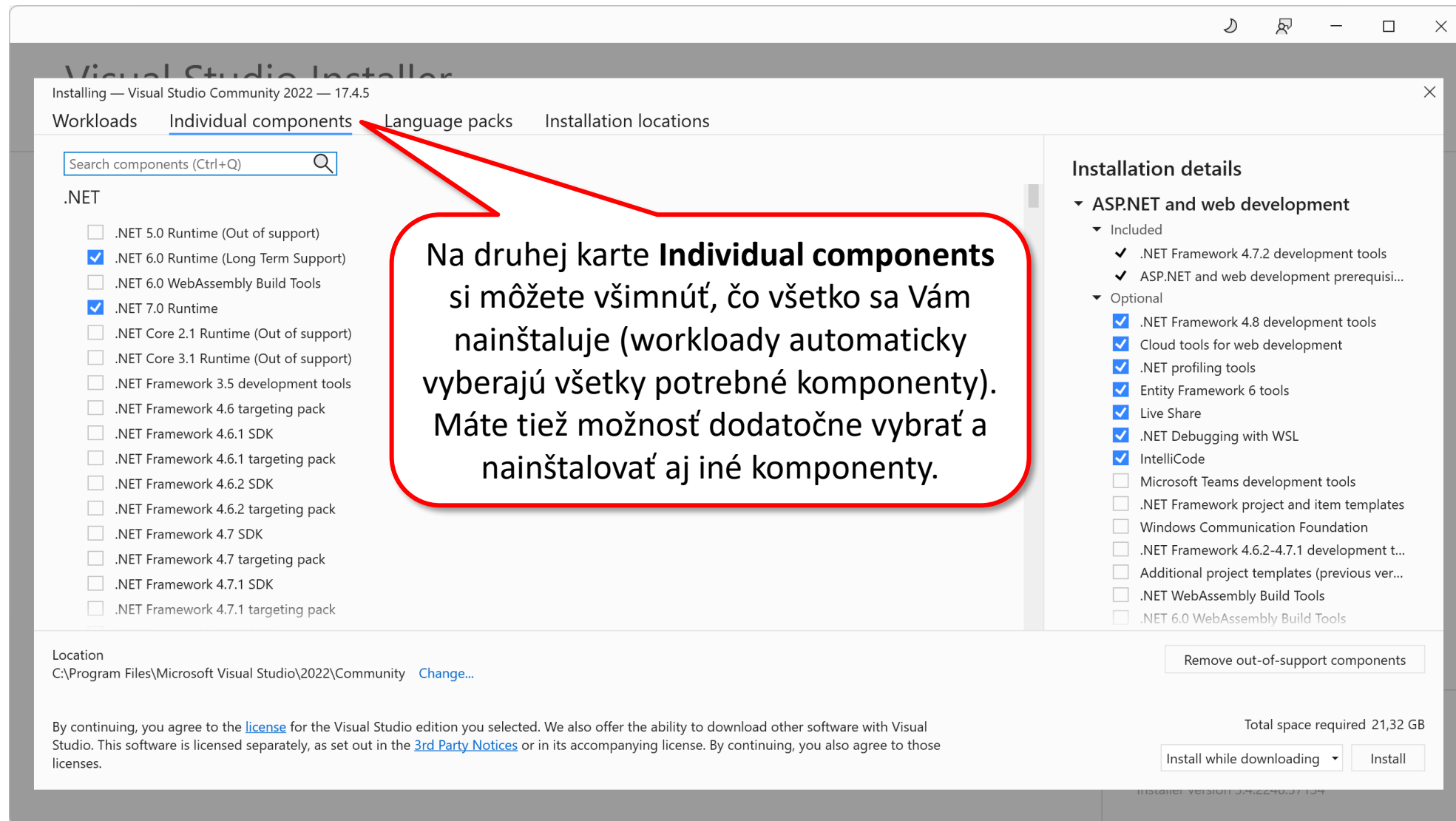
Visual Studio – manuálna inštalácia (1)



Visual Studio – manuálna inštalácia (2)

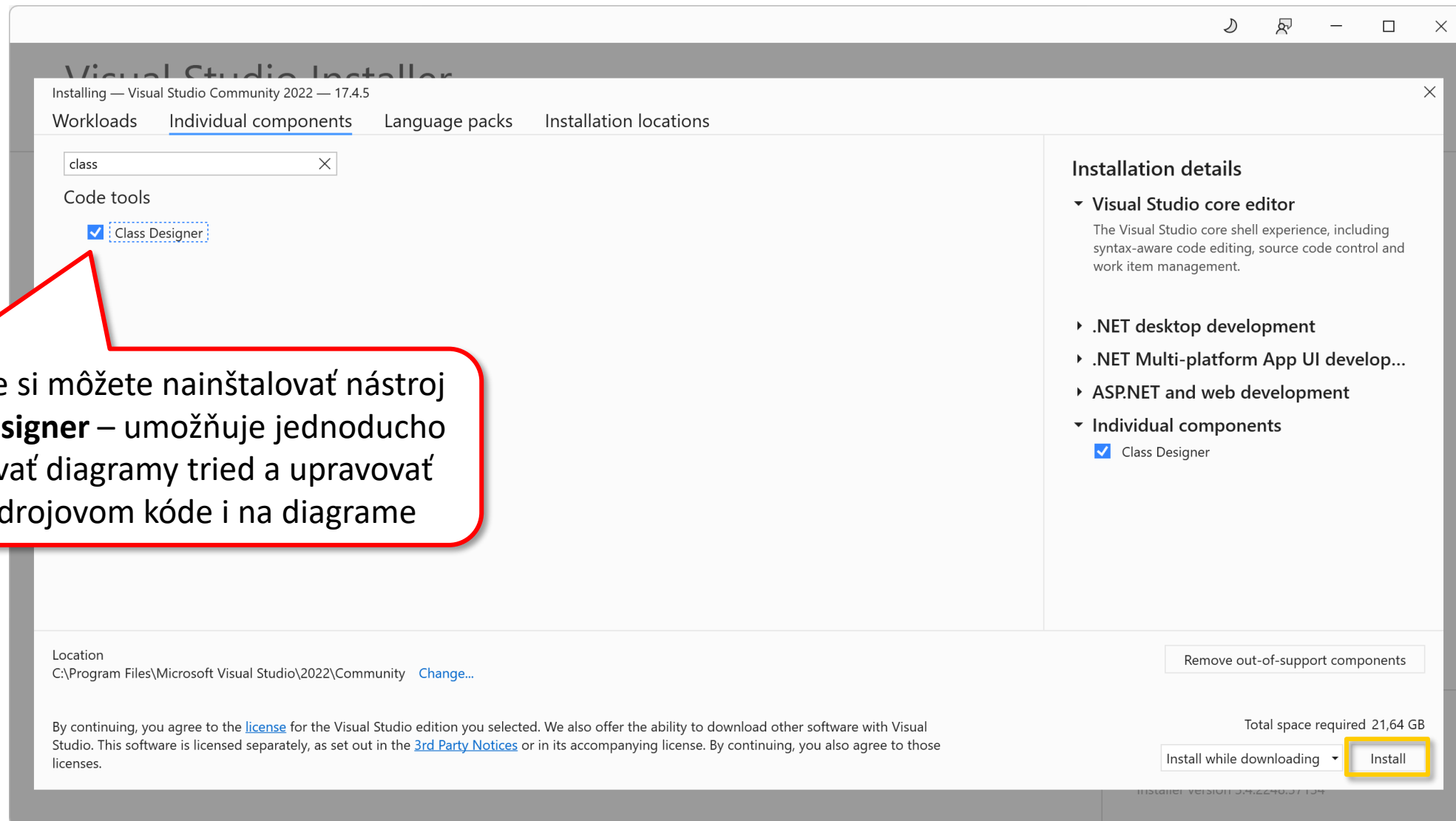


Visual Studio – manuálna inštalácia (3)



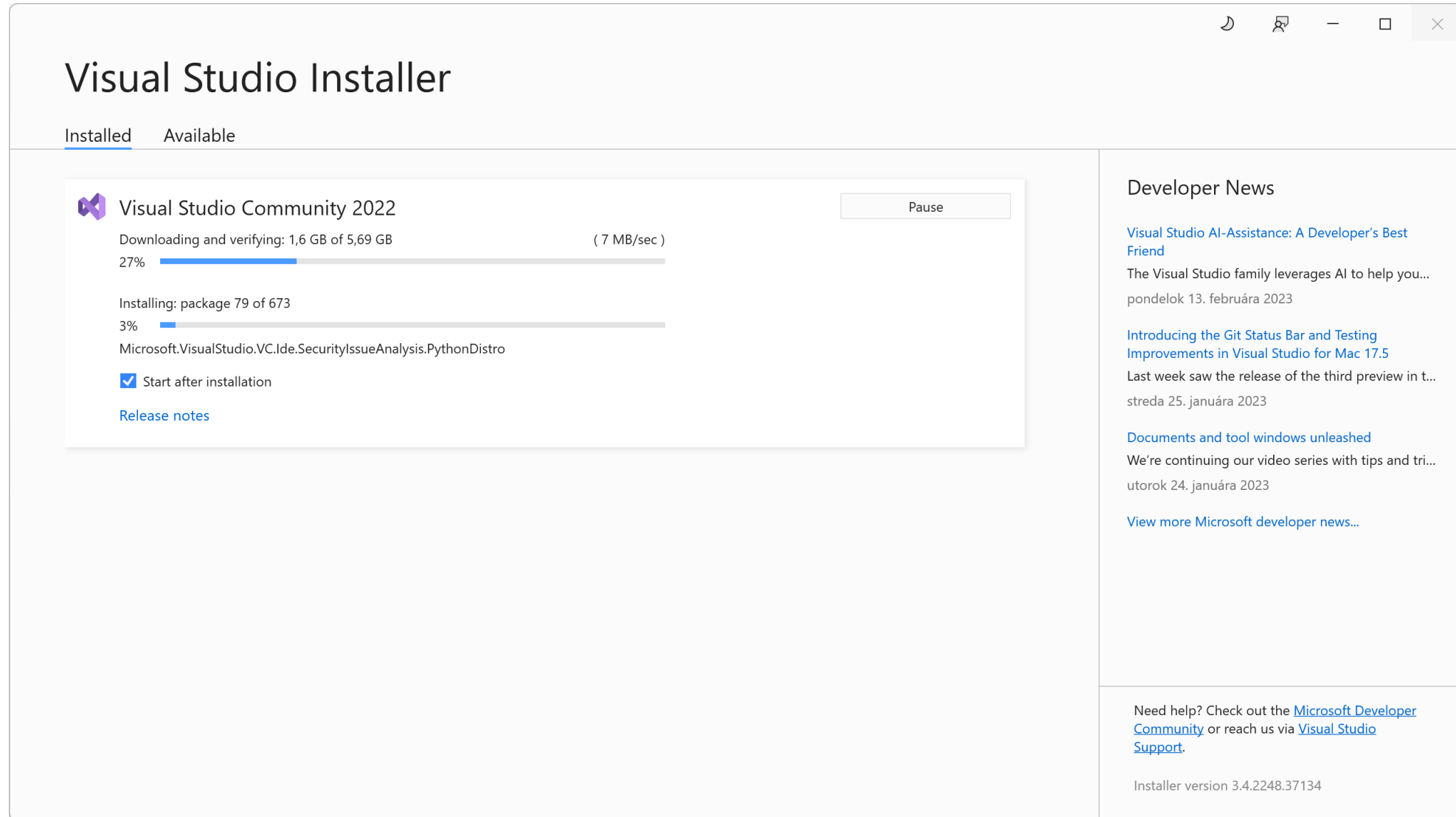
Na druhej karte **Individual components** si môžete všimnúť, čo všetko sa Vám nainštaluje (workloady automaticky vyberajú všetky potrebné komponenty). Máte tiež možnosť dodatočne vybrať a nainštalovať aj iné komponenty.

Visual Studio – manuálna inštalácia (4)



Voliteľne si môžete nainštalovať nástroj **Class Designer** – umožňuje jednoducho generovať diagramy tried a upravovať ich v zdrojovom kóde i na diagrame

Visual Studio – manuálna inštalácia (5)



Visual Studio – manuálna inštalácia (6)

Visual Studio Installer

Installed Available

All installations are up to date.

Visual Studio Community 2022

17.4.5

Powerful IDE for developers, open-source contributors, and individuals

[Release notes](#)

Modify

Launch

More ▾

Developer News

[Visual Studio AI-Assistance: A Developer's Best Friend](#)

The Visual Studio family leverages AI to help you...

pondelok 13. februára 2023

[Introducing the Git Status Bar and Testing Improvements in Visual Studio for Mac 17.5](#)

Last week saw the release of the third preview in...

streda 25. januára 2023

[Documents and tool windows unleashed](#)

We're continuing our video series with tips and tri...

utorok 24. januára 2023

[View more Microsoft developer news](#)

Done installing

Visual Studio has been successfully installed. We recommend rebooting soon to clean up any remaining files.

OK

Installer version 3.4.2248.37134

1

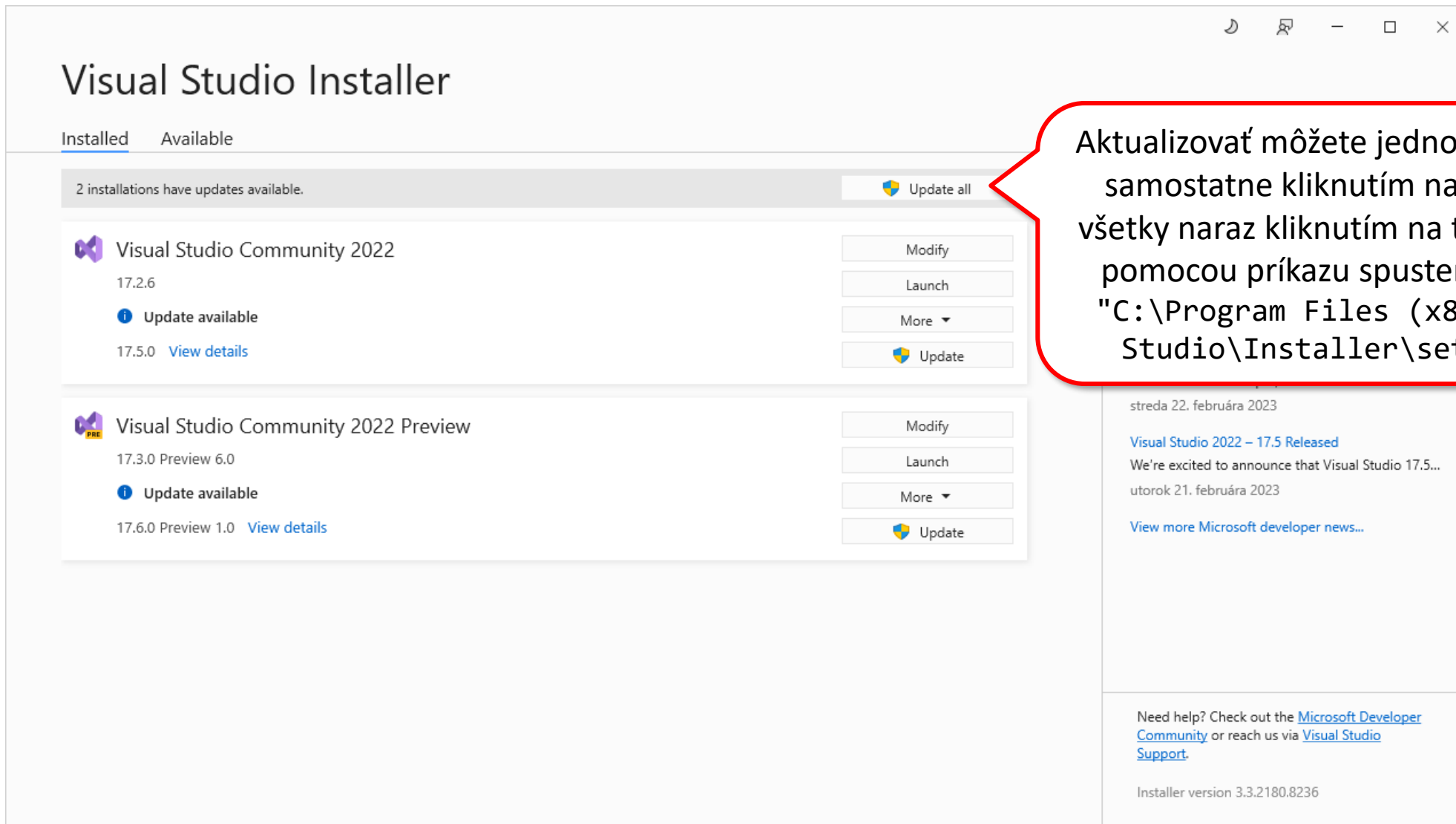
2

Tu sú k dispozícii všetky dostupné verzie Visual Studia, vrátane Preview verzií.

Túto aplikáciu inštalátora môžete v budúcnosti vyvolať buď tak, že vyhľadáte odkaz **Visual Studio Installer** cez Štart alebo priamo vo Visual Studiu otvoríte menu **Tools** a vyberiete možnosť **Get Tools and Features...**

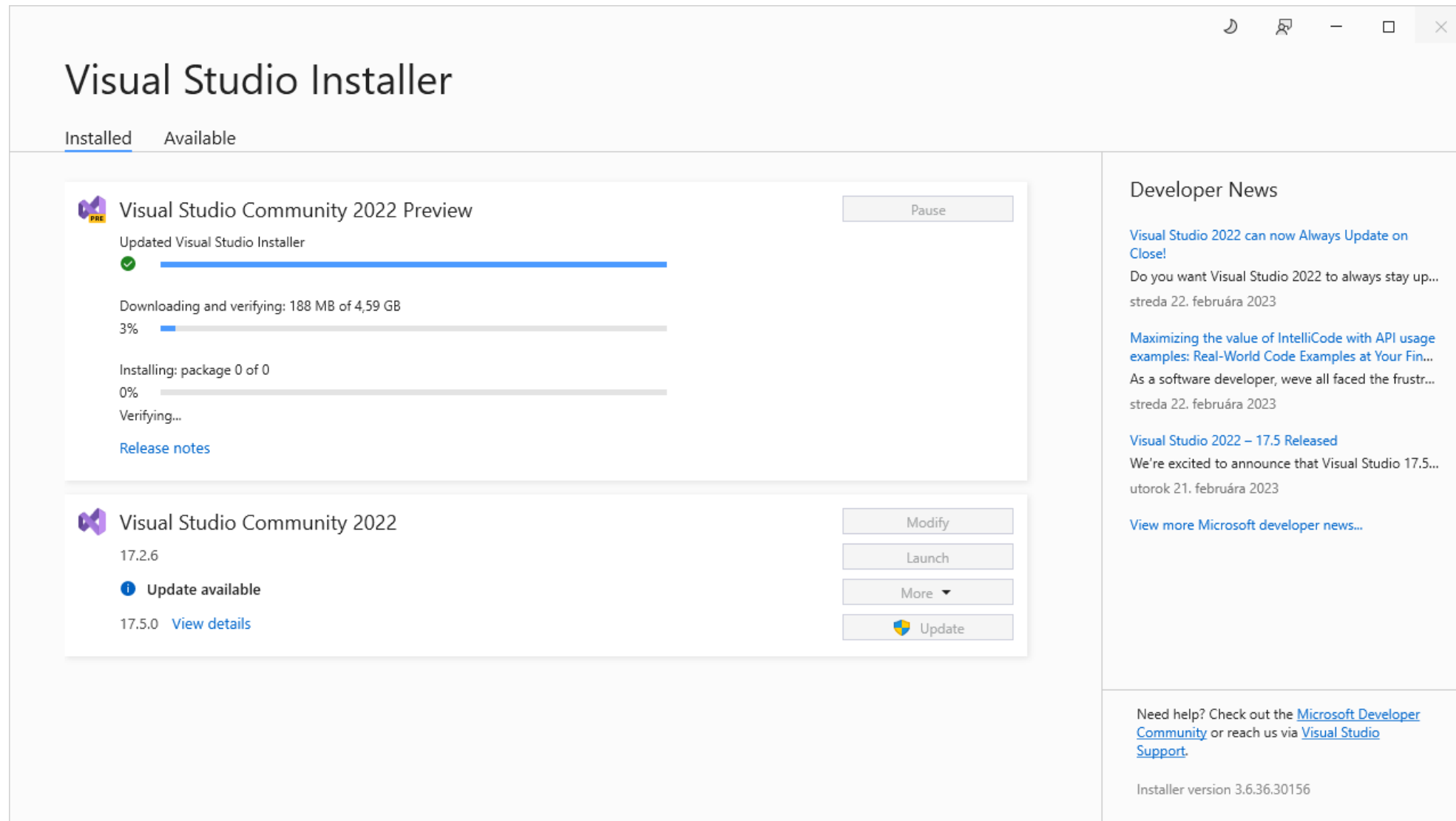
Ak chcete upraviť (pridať alebo odstrániť) workloady alebo komponenty, môžete kliknúť na tlačidlo **Modify**.

Visual Studio – príklad aktualizácie (1)

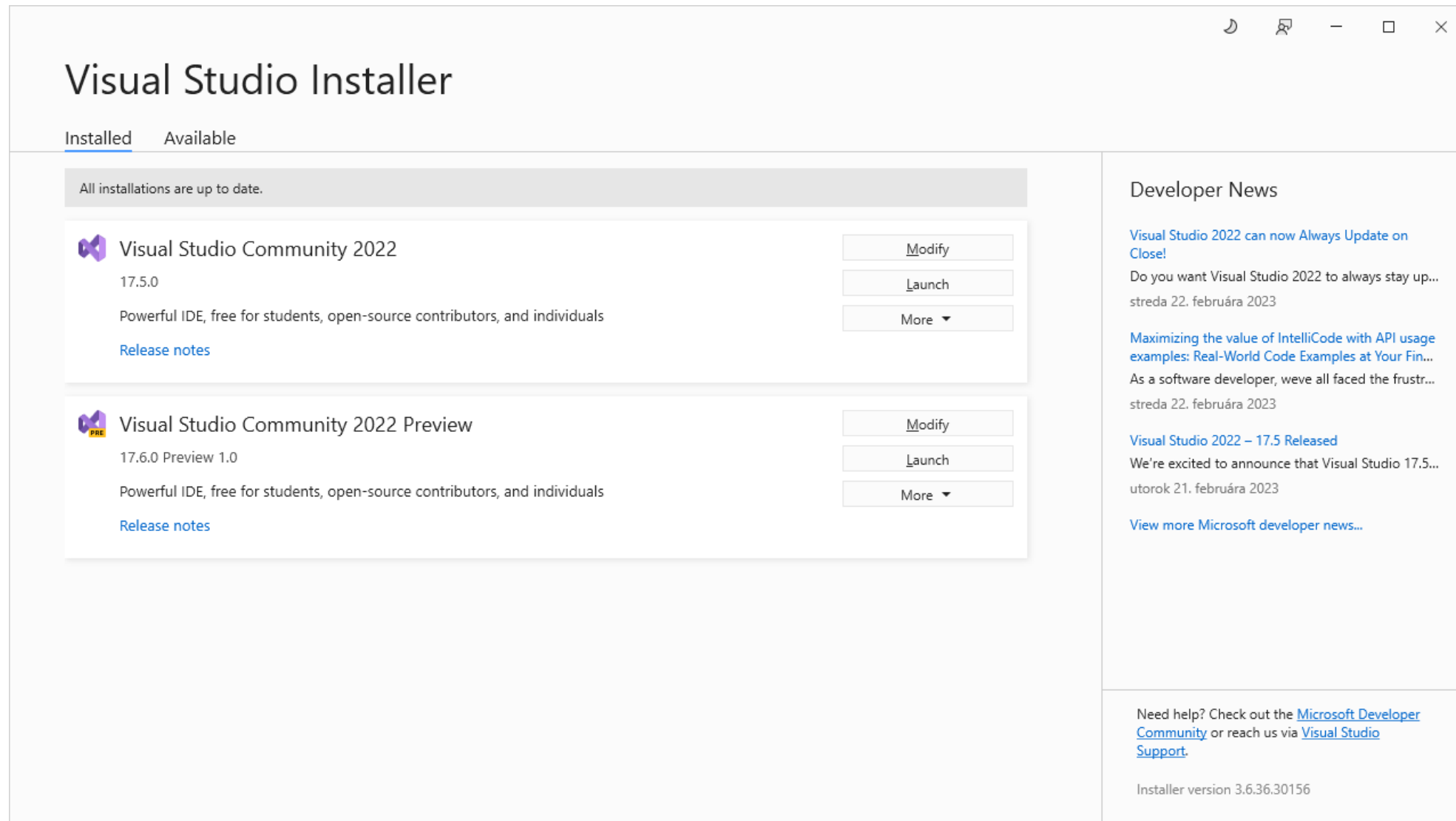


Aktualizovať môžete jednotlivé verzie Visual Studia samostatne kliknutím na tlačidlo **Update** alebo všetky naraz kliknutím na tlačidlo **Update all** alebo pomocou príkazu spusteného s admin právami: "C:\Program Files (x86)\Microsoft Visual Studio\Installer\setup.exe" updateall

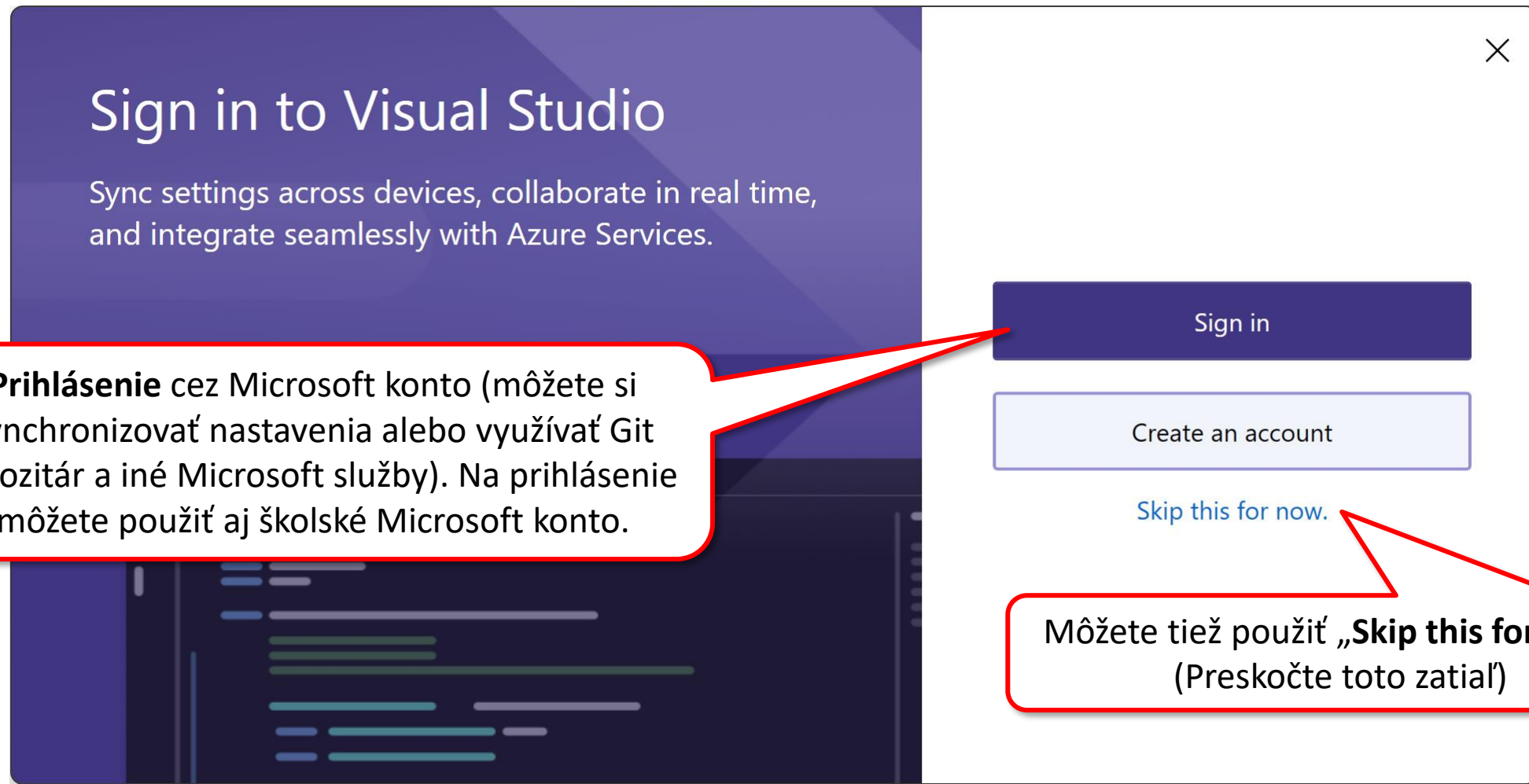
Visual Studio – príklad aktualizácie (2)



Visual Studio – príklad aktualizácie (3)



Visual Studio – prvé spustenie (1)



Visual Studio – prvé spustenie (2)

Personalize your Visual Studio experience

Optimize the layout and keyboard shortcuts for your workflow. Select the color theme that fits your style.

You can always change these settings later.

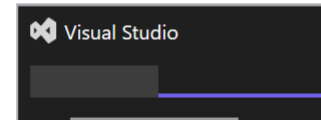
Vyberte „**Visual C#**“ – Visual Studio sa prispôsobí na programovanie v jazyku C# (menu, klávesové skratky, hlavné projekty apod.). Ak ste pri prvom spustení nezvolili *Visual C#* alebo chcete neskôr zmeniť za iné nastavenia, môžete tak urobiť cez menu **Tools** → **Import and Export Settings...** → **Reset all settings** a v 3. kroku zvolíte **Visual C#**

Development settings

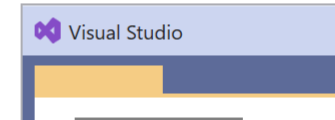
Visual C#

Choose your color theme

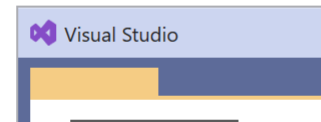
☒ Dark



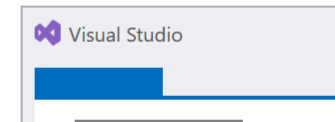
☐ Blue



☐ Blue (Extra Contrast)



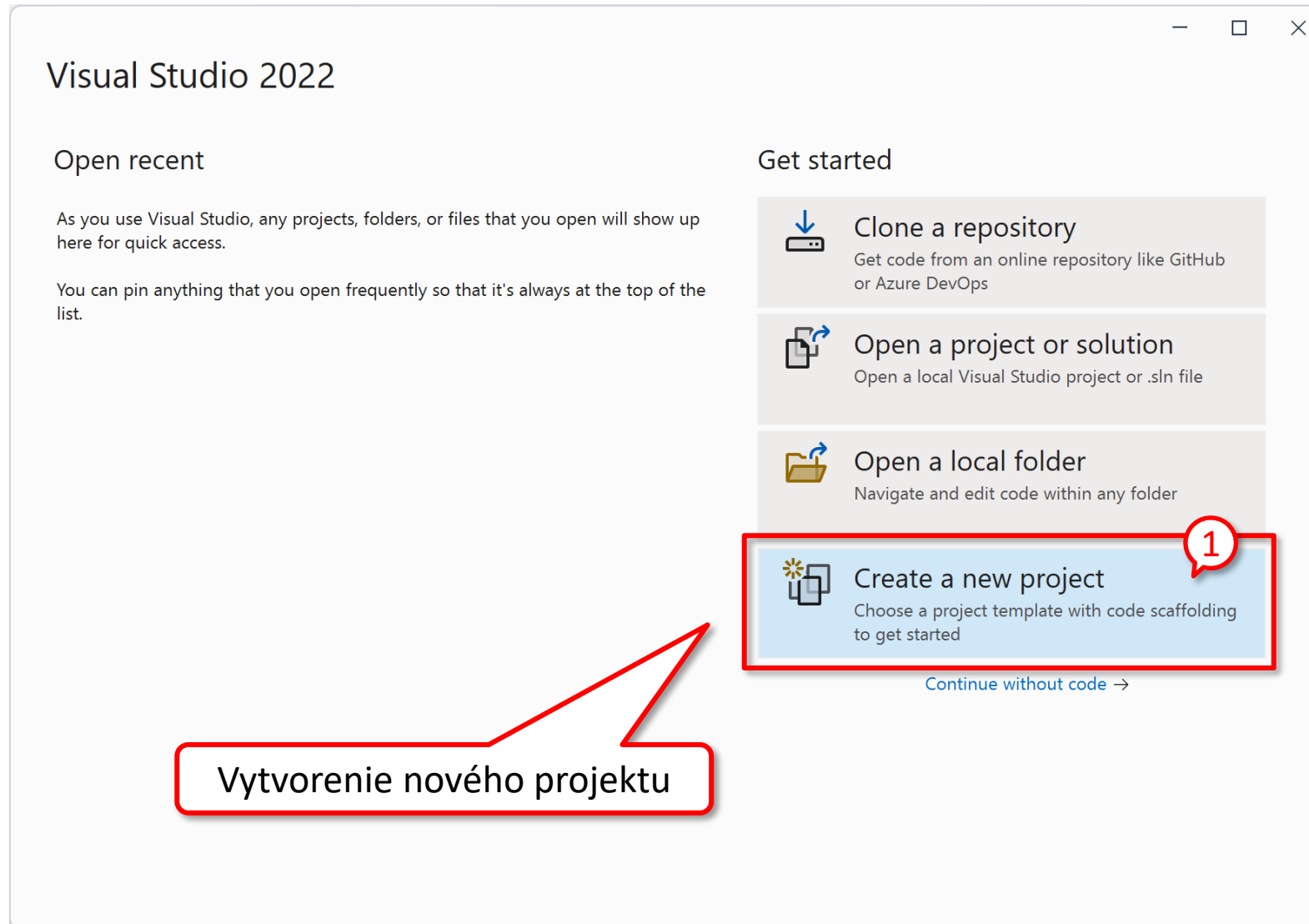
☐ Light



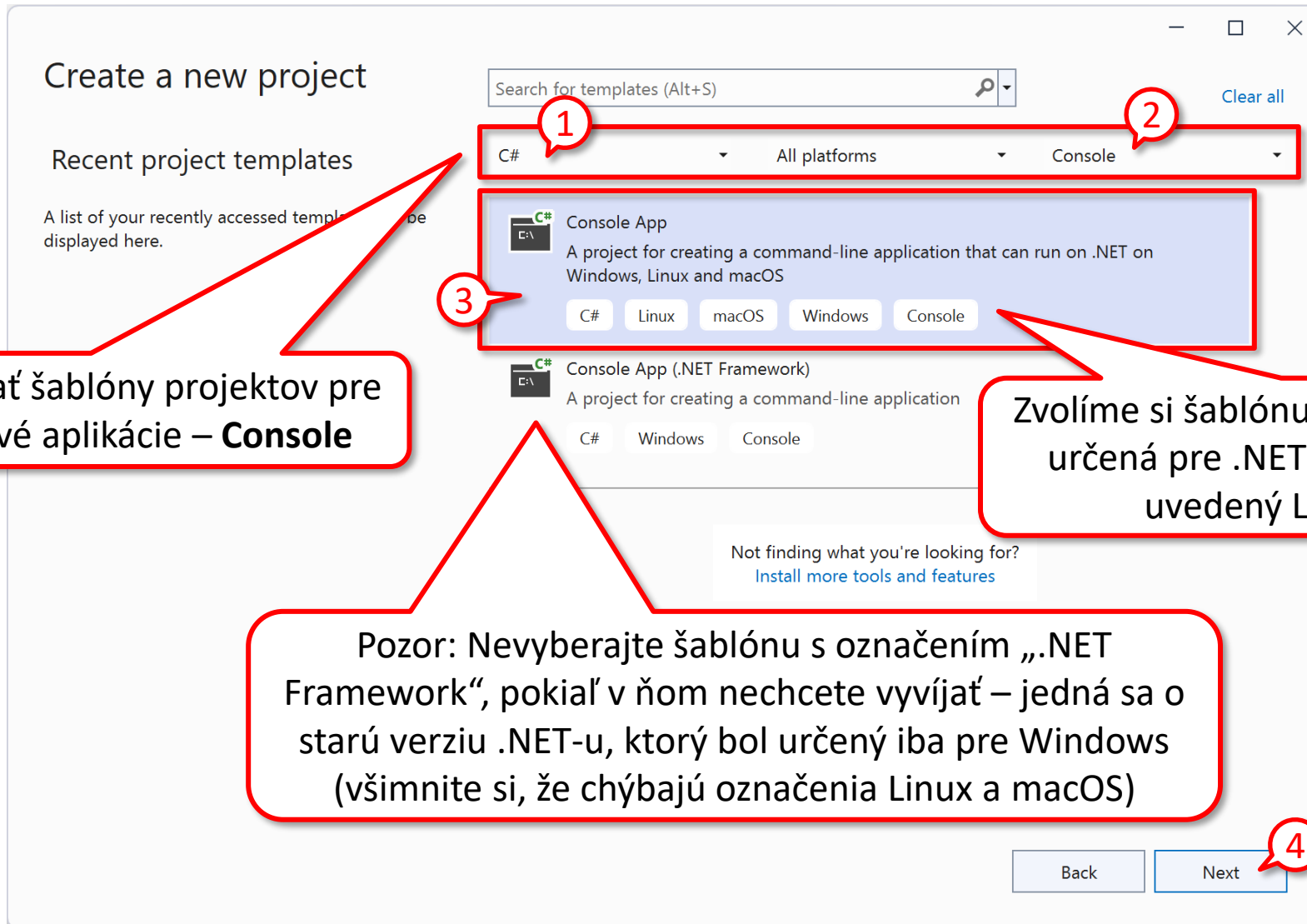
2

Start Visual Studio

Visual Studio – vytvorenie projektu (1)



Visual Studio – vytvorenie projektu (2)



Môžeme si vyfiltrovať šablóny projektov pre jazyk **C#** a konzolové aplikácie – **Console**

Zvolíme si šablónu **Console App**, ktorá je určená pre .NET (všimnite si, že je tu uvedený Linux a macOS)

Pozor: Nevyberajte šablónu s označením „.NET Framework“, pokiaľ v ňom nechcete vyvíjať – jedná sa o starú verziu .NET-u, ktorý bol určený iba pre Windows (všimnite si, že chýbajú označenia Linux a macOS)

Visual Studio – vytvorenie projektu (3)

Názov projektu

Umiestnenie
riešenia

Názov riešenia
("kontajner"
pre projekty)

Configure your new project

Console App C# Linux macOS Windows Console

Project name 1
HelloWorld

Location 2
C:\CSharp\

Solution name 3
Lecture01

☐ Place solution and project in the same directory

Back Next 4

Visual Studio – vytvorenie projektu (4)

Výber **verzie**
cieľového
frameworku
pre projekt (dá
sa zmeniť aj
neskôr)

Additional information

Console App C# Linux macOS Windows Console

Framework ⓘ

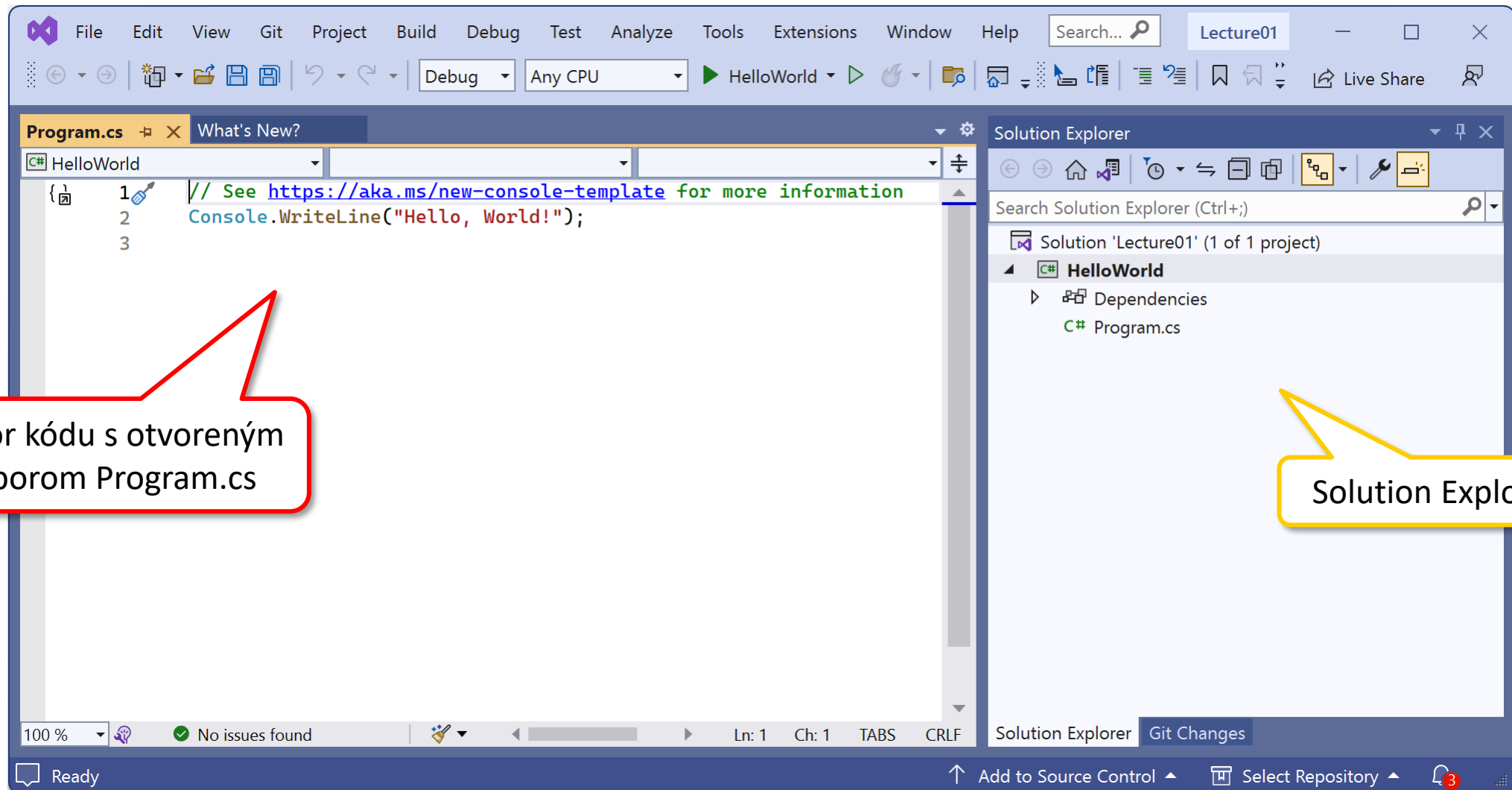
.NET 7.0 (Standard Term Support) 1

☐ Do not use top-level statements ⓘ

Back Create 2

Začiarknite, ak chcete explicitne vygenerovať triedu Program a metódu Main. Ak to nezačiarknete, použije sa štruktúra programu s využitím príkazov na najvyššej úrovni (top-level statements), vtedy kompilátor implicitne sám vygeneruje triedu Program a metódu Main, do ktorej vloží obsah zdrojového kódu zo súboru Program.cs

Visual Studio – vytvorenie projektu (5)

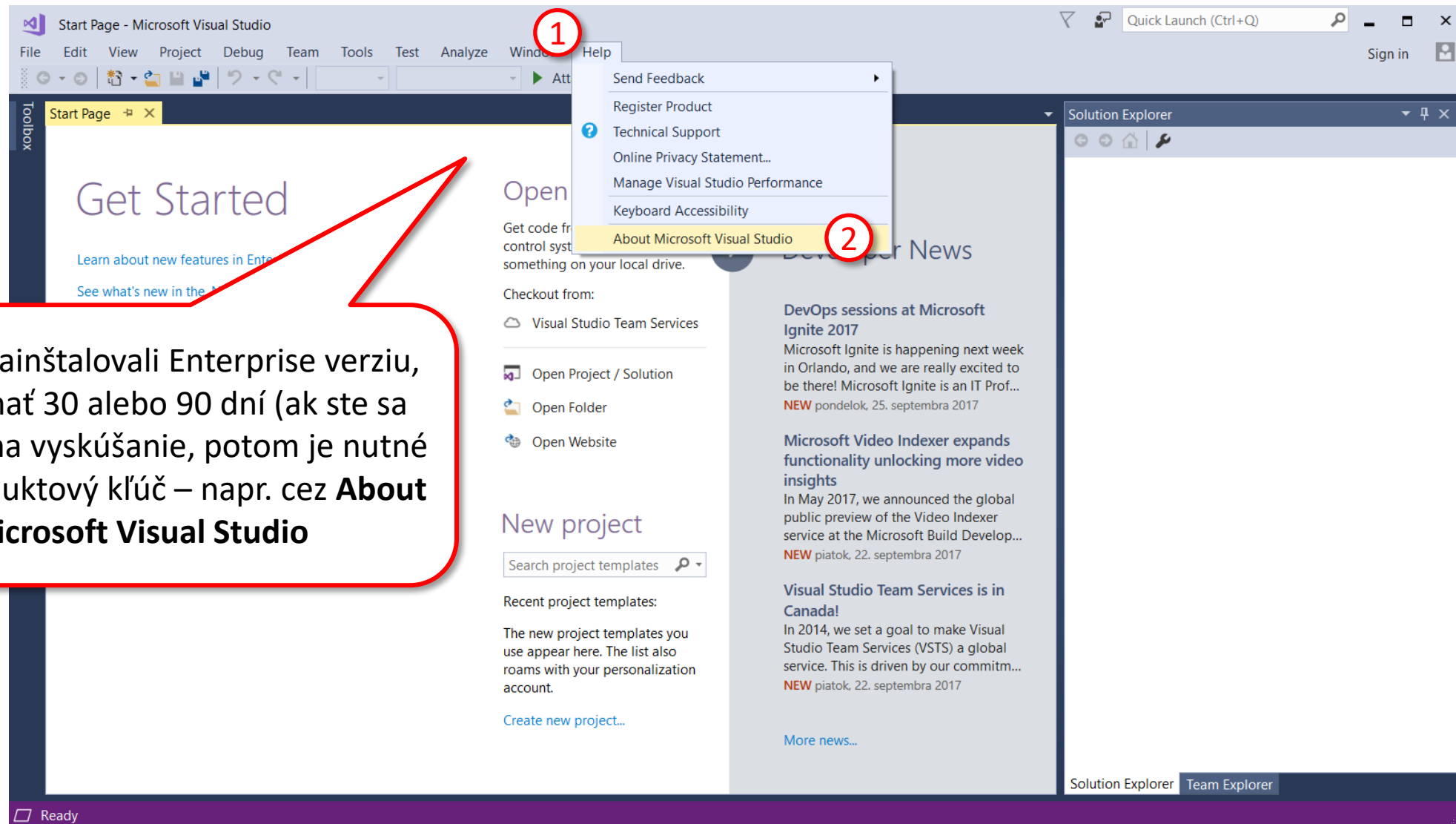


Editor kódu s otvoreným
súborom Program.cs

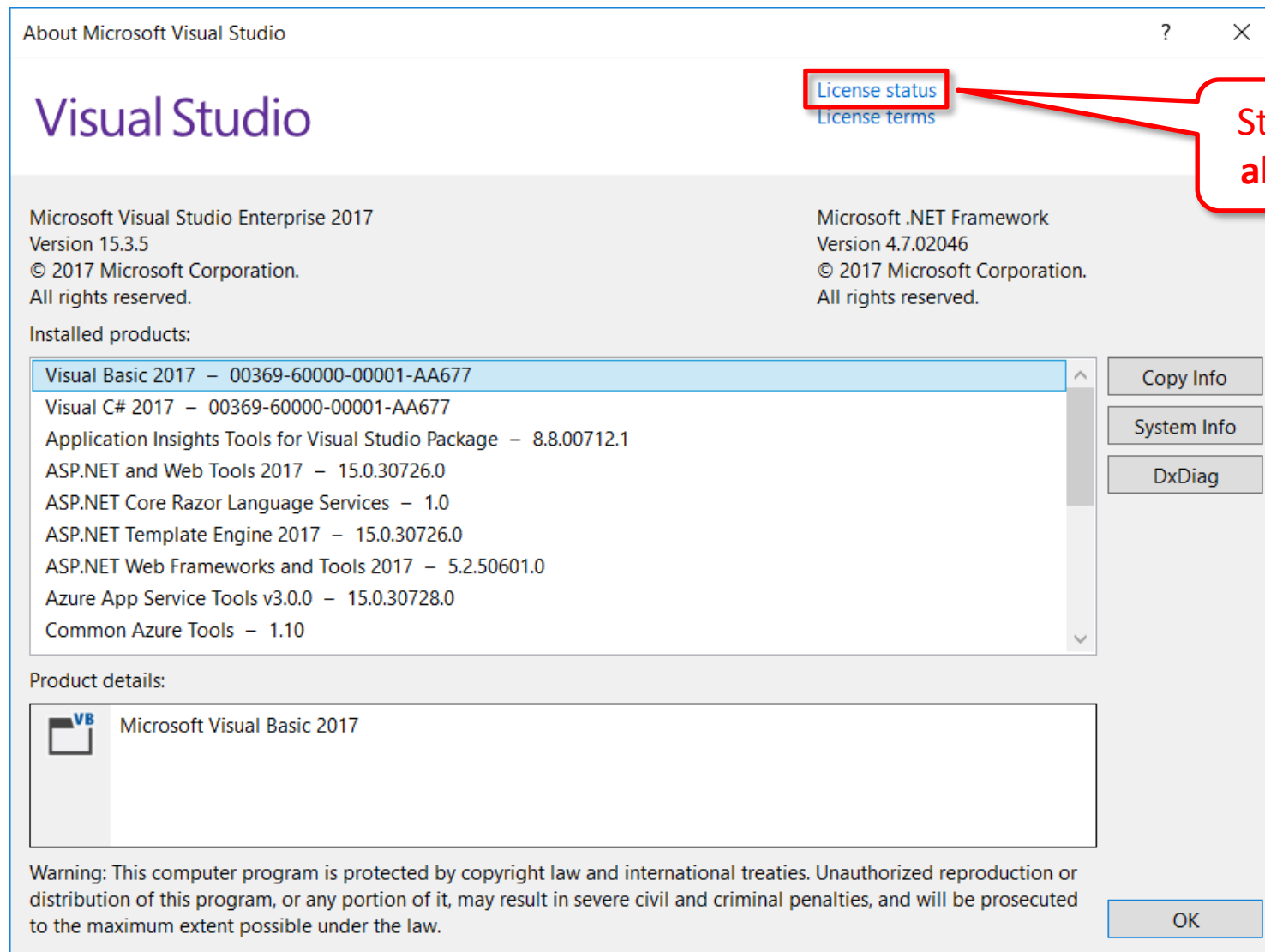
Solution Explorer

Visual Studio verzie Enterprise – aktivácia (1)

Ak ste si nainštalovali Enterprise verziu, budete mať 30 alebo 90 dní (ak ste sa prihlásili) na vyskúšanie, potom je nutné zadať produktový kľúč – napr. cez **About Microsoft Visual Studio**



Visual Studio verzie Enterprise – aktivácia (2)



Visual Studio verzie Enterprise – aktivácia (3)

Sign in to Visual Studio

Visual Studio will help you plan projects, collaborate with your team, and manage your code online from anywhere.

[Learn more](#)

Sign in to start using your Azure credits, publish code to a private Git repository, sync your settings, and unlock the IDE.

[Sign in](#) Don't have an account? [Sign up](#)

All Accounts

[Add an account...](#)

Visual Studio

Enterprise 2017

License: 30 day trial (for evaluation purposes only)
Your evaluation period will end on štvrtok, 26. októbra 2017.

[Check for an updated license](#)

[Unlock with a Product Key](#)

Enter a product key

The 25 character product key is available inside the Microsoft Visual Studio package or on the MSDN subscriber site.

Product key:

[Apply](#)

[Close](#)

Pre aktivovanie a zrušenie
30 dňovej skúšobnej doby zadajte
produktový kľúč (v prípade verzie
Enterprise).

Ak používate verziu Community,
prihláste sa Microsoft kontom.

Ako začať vyvíjať na Linuxe a Macu

- **Linux:**

- Visual Studio Code + rozšírenia + .NET SDK
- JetBrains Rider

- **Mac:**

- Visual Studio Code + rozšírenia + .NET SDK
- JetBrains Rider

- Viac informácií:

- <https://learn.microsoft.com/en-us/dotnet/core/install/>

Learn / .NET / Fundamentals /

Install .NET on Windows, Linux, and macOS

Learn about installing .NET on Windows, Linux, and macOS. Discover the dependencies required to develop, deploy, and run .NET apps.

Windows

OVERVIEW

[Install on Windows](#)

[Supported Windows releases](#)

[Dependencies](#)

[Install with Visual Studio](#)

[Install alongside Visual Studio Code](#)

Q&A

GET STARTED

[Standalone installers](#)

[Visual Studio installers](#)

[Linux feeds](#)

[Docker](#)

[Enterprise deployment](#)

macOS

OVERVIEW

[Install on macOS](#)

[Supported macOS releases](#)

[Install alongside Visual Studio Code](#)

Linux

OVERVIEW

[Linux overview](#)

[Alpine](#)

[CentOS](#)

[Debian](#)

[Fedora](#)

[openSUSE](#)

[Red Hat Enterprise Linux and CentOS Stream](#)

[SUSE Linux Enterprise Server](#)

[Ubuntu](#)

Ako začať vyvíjať cez VS Code

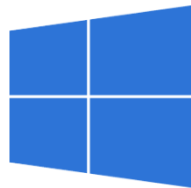
- Nainštalovať **Visual Studio Code** (VS Code) a potrebné nasledujúce **rozšírenia**:
 - C# (ms-dotnettools.csharp) – nutné, aby ste mohli pracovať so C# kódom
 - C# Dev Kit (ms-dotnettools.csdevkit) – rozširuje
- Potrebné nainštalovať **.NET SDK**, pokiaľ ste si nenainštalovali Visual Studio 2022 (s ním sa už .NET SDK nainštaloval)
 - Viac informácií pre vývoj v .NET:
<https://code.visualstudio.com/docs/languages/dotnet>

VS Code – stiahnutie

- <https://code.visualstudio.com/download>

Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.



↓ Windows

Windows 8, 10, 11

User Installer	x64	x86	Arm64
System Installer	x64	x86	Arm64
.zip	x64	x86	Arm64
CLI	x64	x86	Arm64



↓ .deb

Debian, Ubuntu

↓ .rpm

Red Hat, Fedora, SUSE

.deb	x64	Arm32	Arm64
.rpm	x64	Arm32	Arm64
.tar.gz	x64	Arm32	Arm64
Snap	Snap Store		
CLI	x64	Arm32	Arm64

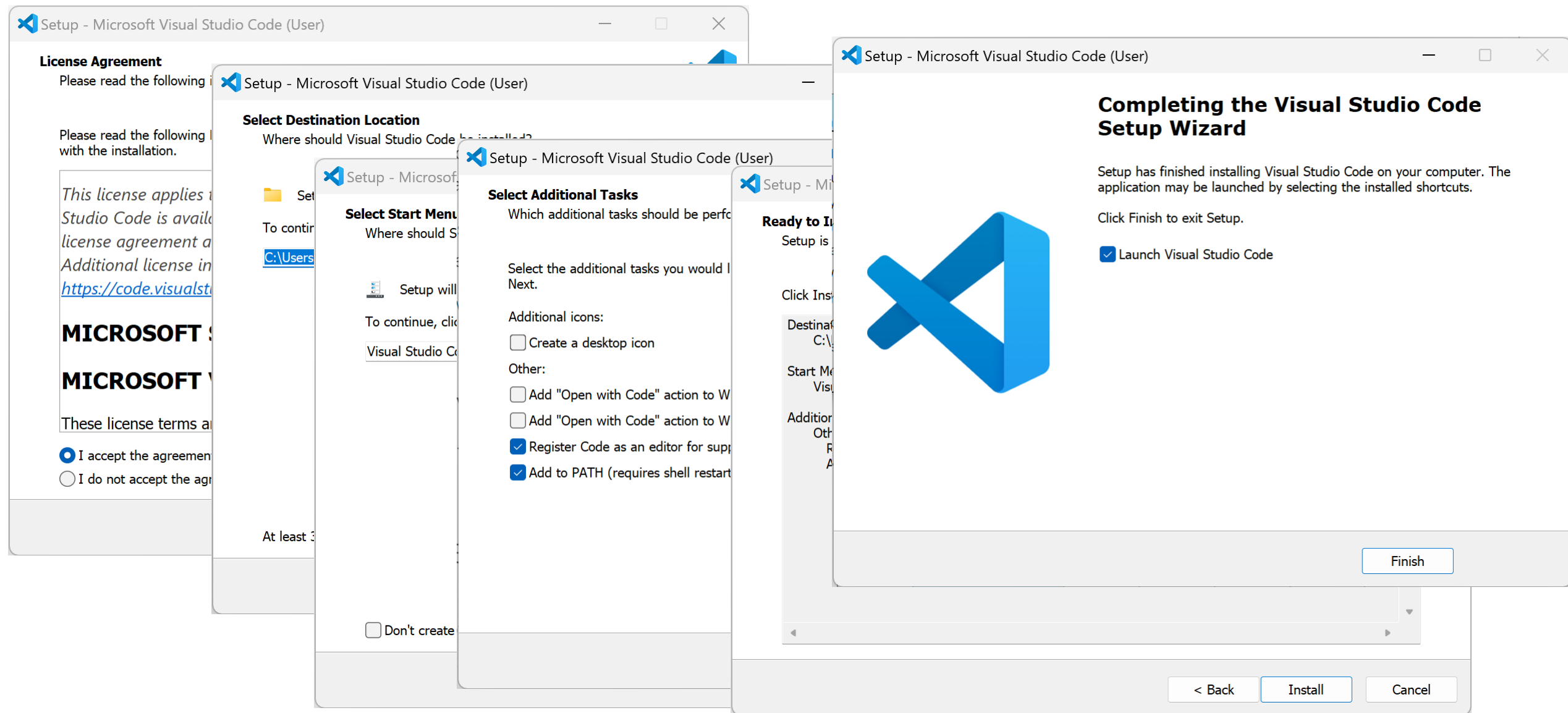


↓ Mac

macOS 10.11+

.zip	Intel chip	Apple silicon	Universal
CLI	Intel chip	Apple silicon	

VS Code – inštalácia



.NET CLI (Command-Line Interface) (1)

- Multiplatformový **nástroj na vývoj, vytváranie, spúšťanie a publikovanie .NET aplikácií**
- Základom je riadiaci program **dotnet**, ktorý sa používa nasledovne:
 - **dotnet** aplikacia-zavisla-na-frameworku.dll
 - spustí .NET aplikáciu publikovanú ako *framework-dependent*, príklad: `dotnet HelloWorld.dll`
 - **dotnet** **príkaz** **argumenty** **voľby**
 - Vytvorí, zostaví, spustí a publikuje aplikáciu v závislosti od zvoleného príkazu (**command**), voliteľných argumentov (**arguments**) a volieb (**options**)
 - Príklady:
 - `dotnet --info`
 - `dotnet --help`
 - `dotnet new`
 - `dotnet new --help`
 - `dotnet new list --language "C#"`
 - `dotnet new console`
 - `dotnet new console --output HelloWorldApp`
 - `dotnet add package Figgie`
 - `dotnet run`
 - Ľubovoľný príkaz si môžete najlepšie pozrieť cez príkaz **help** alebo voľbou **--help** / **-h**, alebo v **dokumentácii**:
<https://learn.microsoft.com/en-us/dotnet/core/tools/>

Learn / .NET / Tools and diagnostics / Tools /

dotnet add package

Article • 10/11/2022 • 2 minutes to read • 12 contributors

Feedback

Arguments

PROJECT

Specifies the project file. If not specified, the command searches the current directory for one.

PACKAGE_NAME

The package reference to add.

Options

-f|--framework <FRAMEWORK>

Adds a package reference only when targeting a specific framework.

-?|-h|--help

Prints out a description of how to use the command.

--interactive

This article applies to: ✓ .NET Core 3.1 SDK and later versions

Name

`dotnet add package` - Adds or updates a package reference in a project file.

Synopsis

```
.NET CLI
dotnet add [<PROJECT>] package <PACKAGE_NAME>
[-f|--framework <FRAMEWORK>] [--interactive]
[-n|--no-restore] [--package-directory <PACKAGE_DIRECTORY>]
[--prerelease] [-s|--source <SOURCE>] [-v|--version <VERSION>]

dotnet add package -h|--help
```

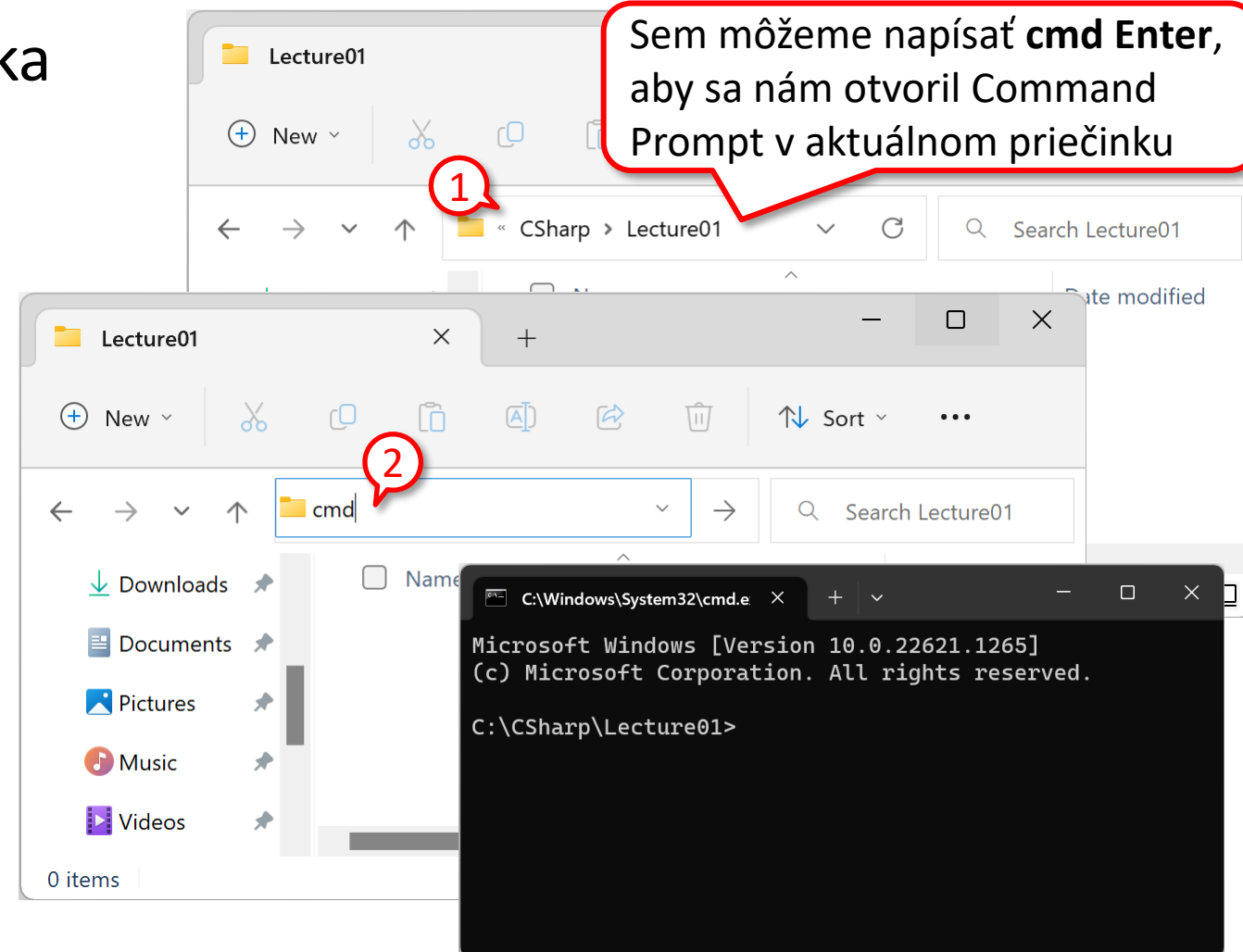
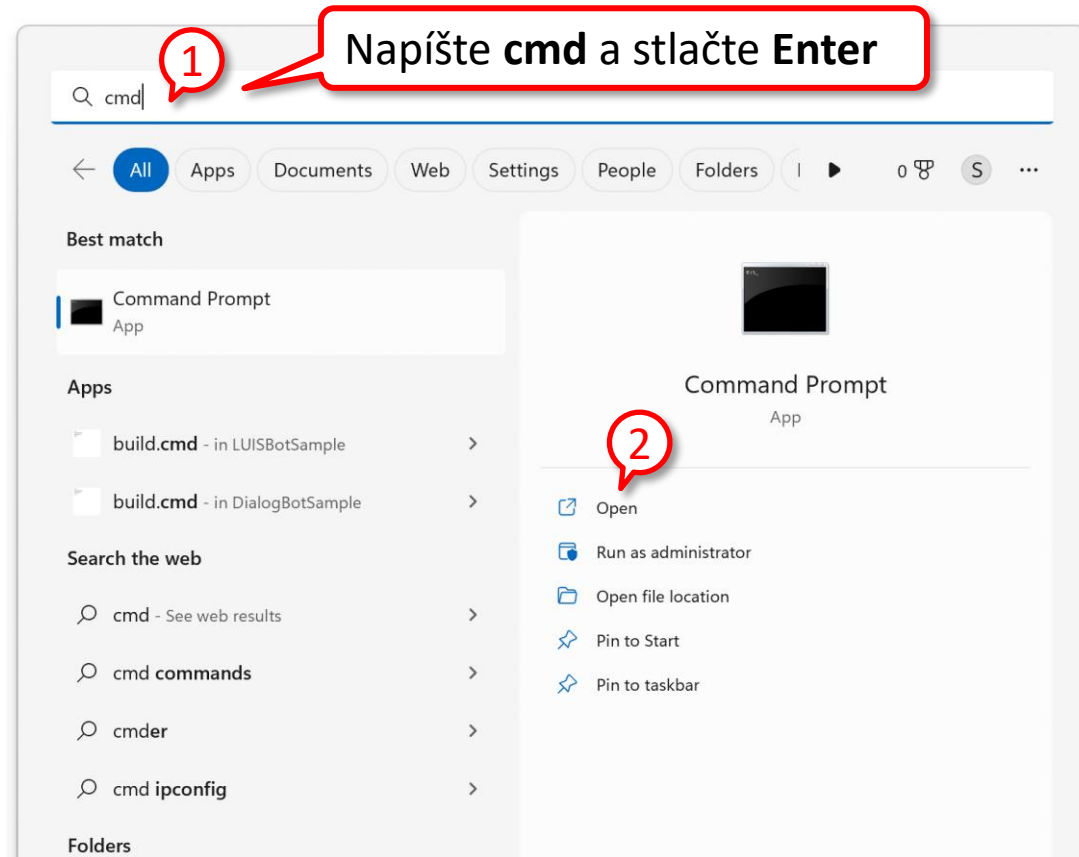
.NET CLI (Command-Line Interface) (2)

- **Ďalšie základné príkazy:**

- **--help** alebo **-h** (skratka) alebo **príkaz --help** alebo **príkaz -h** – vypíše popis použitia programu / príkazu
 - Napr. **dotnet --help** zobrazí popis použitia programu dotnet, **dotnet new --help** zobrazí popis použitia príkazu new
- **--info**, **--version**, **--list-sdks**, **--list-runtimes** – zobrazí rôzne informácie o .NET inštalácii, verziach .NET SDK a runtimeov
- **help príkaz** – zobrazuje podrobnejšiu dokumentáciu k príkazu online vo webovom prehliadači
 - Napr. **dotnet help run** otvorí webovú stránku dokumentácie k príkazu „dotnet run“ na stránke learn.microsoft.com
- **new list** – zobrazí zoznam dostupných šablón, ktoré sa môžu použiť pomocou príkazu dotnet new
 - **dotnet new list --language F#** zobrazí zoznam šablón pre jazyk F#
- **new šablóna** – vytvorí nový projekt, konfiguračný súbor alebo riešenie na základe zadanej šablóny
 - Napr. **dotnet new console** vytvorí projekt konzolovej aplikácie s názvom aktuálneho priečinka, **dotnet new classlib** vytvorí projekt knižnice, **dotnet new console --output MojaSuperAppka** vytvorí projekt konzolovej aplikácie do nového priečinka MojaSuperAppka
- **restore** – používa NuGet na obnovenie (stiahnutie) závislostí, ako aj nástrojov špecifických pre projekt, ktoré sú uvedené v súbore projektu; spúšťa sa automaticky po príkazoch new, build, run, test, publish, pack
- **build** – zostaví projekt a všetky jeho závislosti do množiny súborov s príponou .dll spolu s ďalšími vytvorenými súborami, v závislosti od nastavení
- **run** – spustí zdrojový kód bez akýchkoľvek explicitných kompilačných alebo spúšťacích príkazov; ak je to potrebné, najskôr spustí príkaz restore a potom build
 - Napr. **dotnet run** spustí projekt v aktuálnom priečinku
- **publish** – skompiluje aplikáciu, prečíta jej závislosti špecifikované v súbore projektu a publikuje výslednú sadu súborov do priečinka na nasadenie do hostiteľského systému

.NET CLI – otvorenie Command Prompt / Terminal

- Otvorte si príkazový riadok – „cmd“, „terminal“ alebo „powershell“
 - Bud' cez Štart alebo Prieskumníka



.NET CLI cez Terminál na Windows

Príkazom **md** vytvoríme nový priečinok **HelloWorld**, do ktorého chceme vytvoriť nový projekt so zdrojovým kódom.

Príkazom **cd** sa presunieme do novo vytvoreného priečinku HelloWorld.

Príkazom **dotnet new console** z .NET SDK vytvoríme nový projekt konzolovej aplikácie. Ak chcete vytvoriť iný typ projektu, použite iný argument. Ak chcete preskočiť krok vytvárania nového priečinka, zavolajte **dotnet new console --output HelloWorld**

Vytvoril sa nám súbor projektu **HelloWorld.csproj** a zdrojový kód **Program.cs** s kódom pre výpis reťazca *Hello World!* na výstup.

Pre spustenie aplikácie zavoláme príkaz **dotnet run**, inou možnosťou je spustiť program priamo **dotnet bin\Debug\platforma\HelloWorld.dll** alebo **bin\Debug\platforma\HelloWorld.exe** (na Windowse, na iných platformách bez prípony .exe)

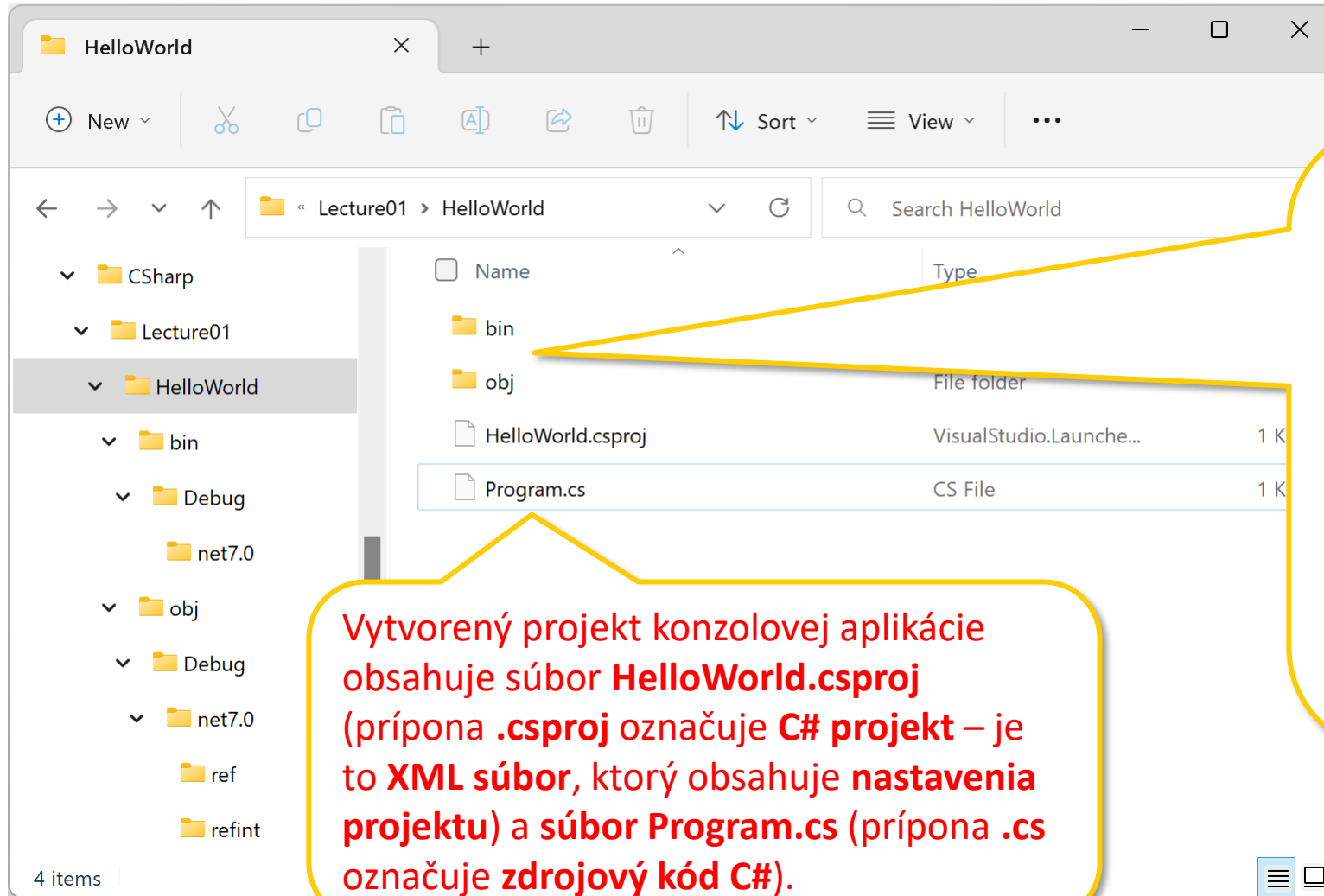
Aplikácia skompilovala, spustila a vypísala na konzolu reťazec Hello World!

```
C:\Windows\System32\cmd.e x
C:\CSharp\Lecture01>md HelloWorld
C:\CSharp\Lecture01>cd HelloWorld
C:\CSharp\Lecture01\HelloWorld>dotnet new console
The template "Console App" was created successfully.

Processing post-creation actions...
Restoring C:\CSharp\Lecture01\HelloWorld\HelloWorld.csproj:
  Determining projects to restore...
  Restored C:\CSharp\Lecture01\HelloWorld\HelloWorld.csproj (in 206 ms).
Restore succeeded.

C:\CSharp\Lecture01\HelloWorld>dotnet run
Hello, World!
C:\CSharp\Lecture01\HelloWorld>
```

Čo obsahuje projekt?



Vytvorený projekt konzolovej aplikácie obsahuje súbor **HelloWorld.csproj** (prípona **.csproj** označuje **C# projekt** – je to **XML súbor**, ktorý obsahuje **nastavenia projektu**) a súbor **Program.cs** (prípona **.cs** označuje **zdrojový kód C#**).

Projekt obsahuje aj priečinok **obj**, ktorý obsahuje viaceré textové a objektové súbory. Po vytvorení projektu obsahuje napr. *project.assets.json* – JSON súbor obsahujúci zoznam závislostí a verzií knižníc. Druhý priečinok **bin** sa vytvára tiež po príkaze *build* alebo *run*, obsahuje výslednú aplikáciu v konkrétnom priečinku podľa cieľového frameworku (**net8.0**). Obidva priečinky **bin** a **obj** môžeme kedykoľvek zmazať, po príkazoch *build* alebo *run* sa totiž automaticky vytvoria.

HelloWorld.csproj

Typ použitého SDK – základom je `Microsoft.NET.Sdk` pre konzolové a aktuálne desktopové aplikácie. Pre webové aplikácie pozrite iné možnosti v dokumentácii: <https://learn.microsoft.com/en-us/dotnet/core/project-sdk/overview#available-sdks>

Od .NET 6 sa začali používať **implicitné usingy**. Ak je táto hodnota povolená (enable), určité menné priestory sú automaticky začlenené prostredníctvom direktívy `global using` – aké sú to, **záleží od zvoleného Sdk**, napr. pre `Microsoft.NET.Sdk` sú to tieto:

[`System`](#)
[`System.Collections.Generic`](#)
[`System.IO`](#)
[`System.Linq`](#)
[`System.Net.Http`](#)
[`System.Threading`](#)
[`System.Threading.Tasks`](#)

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```

Nullable – kontrola referenčných typov na hodnotu null. Ak je to povolené (enable), všetky referenčné premenné, ak nie sú ich typu deklarované s otáznikom, musia mať hodnotu.

Cieľový framework – hodnotou je tzv. **TFM** (target framework moniker): **net48** (.NET Framework), **netcoreapp3.1** (.NET Core 3.1), **net5.0** (.NET 5), **net6.0** (.NET 6), **net7.0** (.NET 7), , **net8.0** (.NET 8).

K dispozícii sú aj špecifické TFM pre operačné systémy, napr.:
net8.0-windows
net8.0-android

Viac info:

<https://learn.microsoft.com/en-us/dotnet/standard/frameworks>

Aká verzia C# sa použije?

- Kompilátor na základe vybranej verzie cieľového frameworku určuje, aká bude použitá **verzia jazyka C#**:

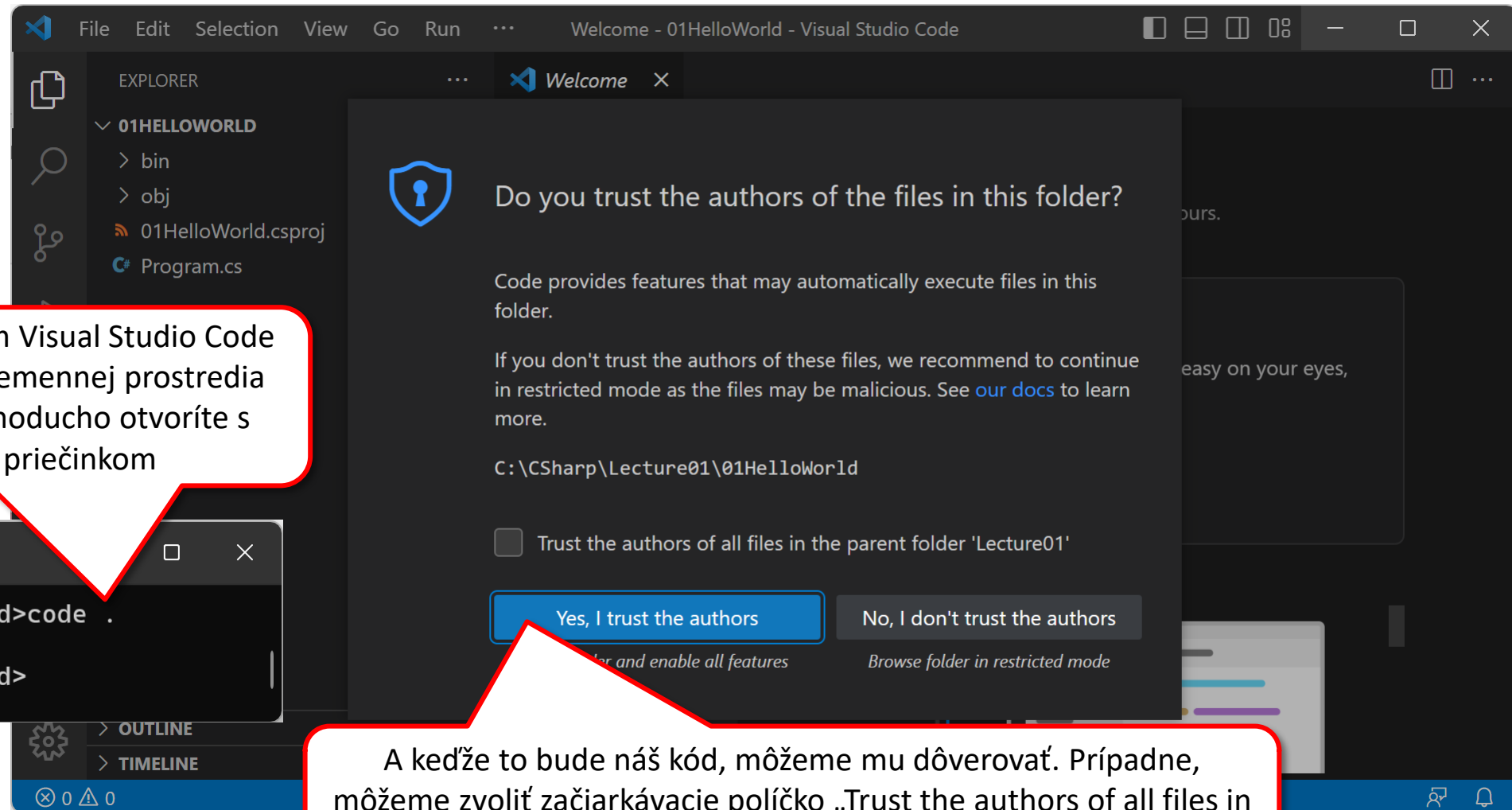
Cieľový framework	Verzia	Implicitná verzia jazyka C#
.NET	8.x	C# 12
.NET	7.x	C# 11
.NET	6.x	C# 10
.NET	5.x	C# 9.0
.NET Core	3.x	C# 8.0
.NET Core	2.x	C# 7.3
.NET Standard	2.1	C# 8.0
.NET Standard	2.0	C# 7.3
.NET Standard	1.x	C# 7.3
.NET Framework	všetky	C# 7.3

Zdroj: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/configure-language-version>

VS Code – otvorenie cez cmd – „code .“

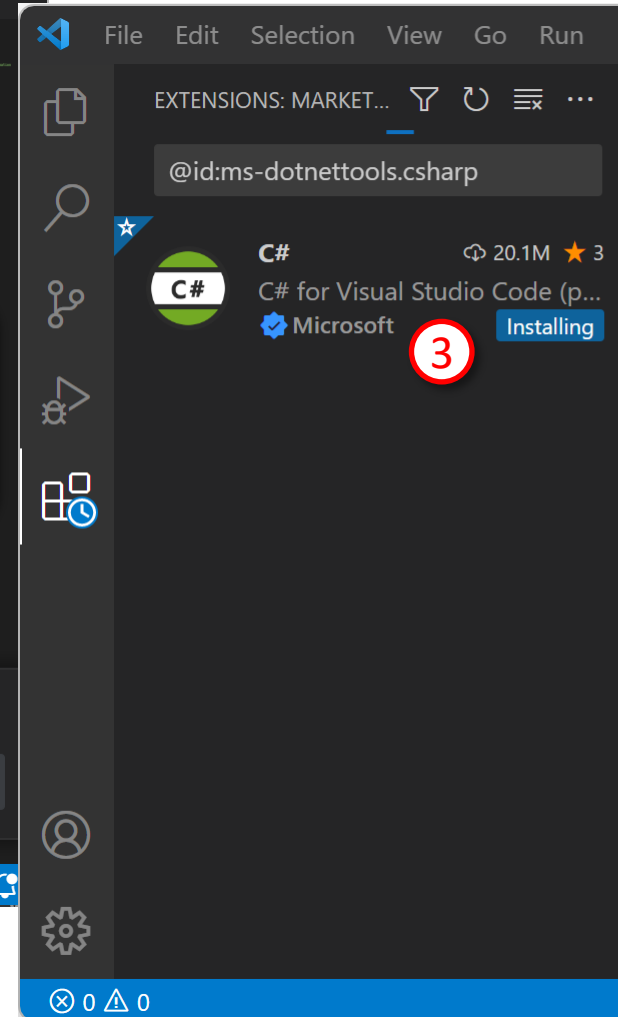
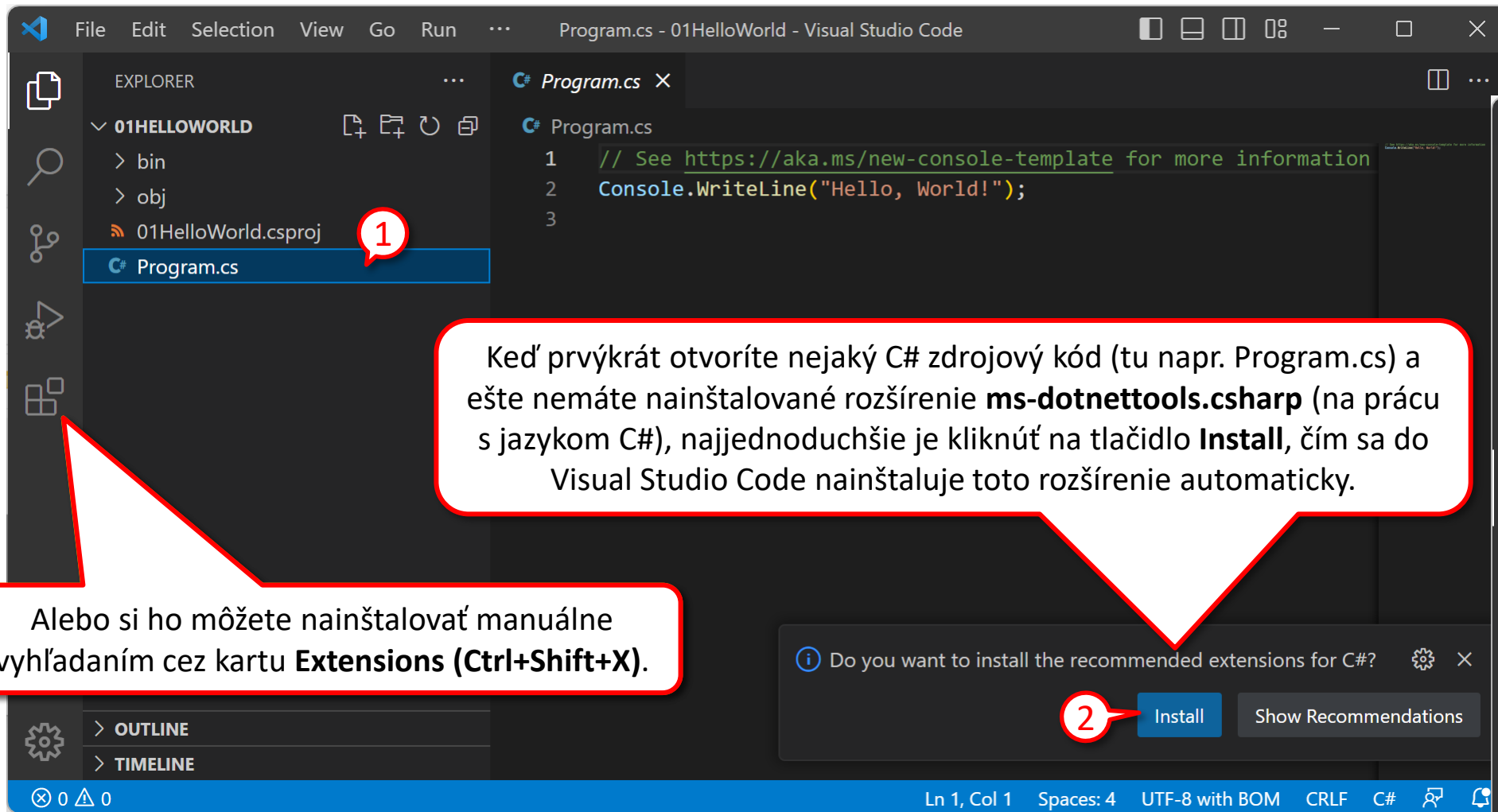
Ak máte nainštalovaný program Visual Studio Code a priradenú cestu k nemu v premennej prostredia Path, príkazom **code .** ho jednoducho otvoríte s otvoreným aktuálnym priečinkom

```
C:\Windows\System32' x + v x
C:\CSharp\Lecture01\HelloWorld>code .
C:\CSharp\Lecture01\HelloWorld>
```



A keďže to bude náš kód, môžeme mu dôverovať. Prípadne, môžeme zvoliť začiarkávacie políčko „Trust the authors of all files in the parent folder“, aby sa dôveroval celý nadradený priečinok

VS Code – automatická inštalácia rozšírenia



Hello World – pred C# 9

- **Stará štandardná štruktúra programu pred C# 9:**

```
using System;
```

Direktívou using sa pridávajú menné priestory do oboru platnosti, tzn. v kóde nemusíme dátovým typom uvádzať ich plné kvalifikované názvy (napr. `System.Console.WriteLine`). Od .NET 6 sú používané implicitné globálne usingy, kde patrí aj menný priestor `System`, takže ho už nemusíme direktívou `using` zavádzať

```
namespace HelloWorld  
{
```

Menný priestor HelloWorld, od C# 10 môžeme deklarovať namespace bez použitia zložených zátvoriek {}, vtedy je definovaný pre všetky typy v súbore

```
    class Program  
    {
```

Trieda Program zapuzdrená do menného priestoru **HelloWorld**

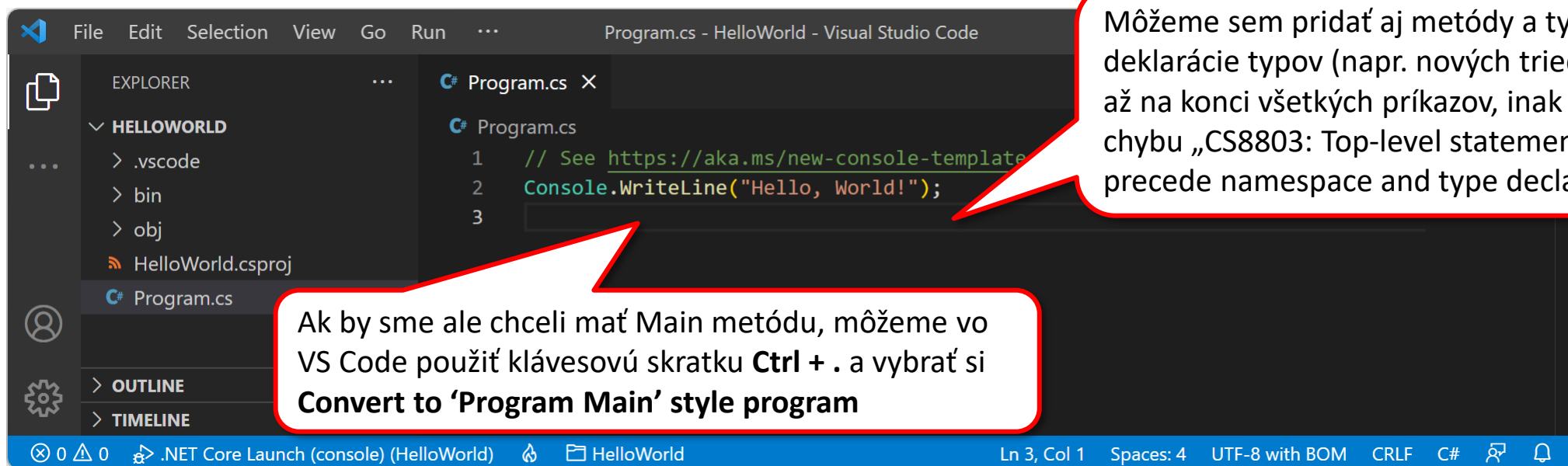
```
        static void Main(string[] args)  
        {  
            Console.WriteLine("Hello World!");  
        }  
    }
```

Statická metóda **Main()** – **vstupný bod aplikácie**; obsahuje parameter **args** predstavujúci pole argumentov, ktoré boli zadane aplikácii pri spustení

```
}
```

Hello World – od C# 10

- **Štandardná štruktúra programu od C# 10 (.NET 6) je s využitím príkazov na najvyššej úrovni (top-level statements)**
 - nie sú potrebné menné priestory, triedy, metóda Main
 - **kompilátor sám vygeneruje triedu Program a metódu Main**, do ktorej vloží obsah zdrojového kódu:



Hello World – od C# 10 (voľba --use-program-main)

```
C:\Windows\System32\cmd.e x + v
C:\CSharp\Lecture01>dotnet new console --output HelloWorldMain --use-program-main
The template "Console App" was created successfully.

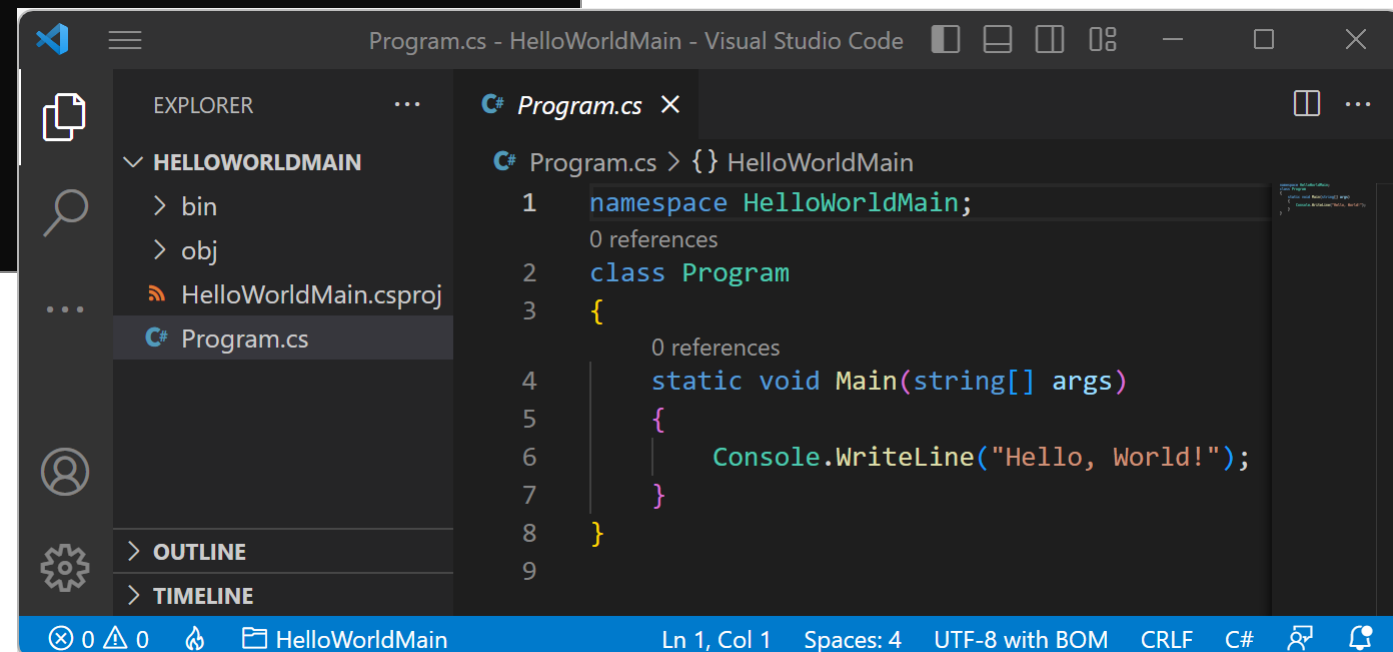
Processing post-creation actions...
Restoring C:\CSharp\Lecture01\HelloWorldMain\HelloWorldMain.csproj:
  Determining projects to restore...
  Restored C:\CSharp\Lecture01\HelloWorldMain\HelloWorldMain.csproj (in 77 ms).
Restore succeeded.

C:\CSharp\Lecture01>cd HelloWorldMain

C:\CSharp\Lecture01\HelloWorldMain>code .

C:\CSharp\Lecture01\HelloWorldMain>
```

Od .NET SDK 6.0.300 môžeme pri vytváraní nového projektu použiť voľbu: **--use-program-main**, čím sa vytvorí štandardná trieda **Program** s metódou **Main()**, t. j. vygeneruje sa nám program bez použitia „top-level statements“.



```
Program.cs - HelloWorldMain - Visual Studio Code
C# Program.cs x
C# Program.cs > {} HelloWorldMain
1 namespace HelloWorldMain;
  0 references
2 class Program
3 {
  0 references
4     static void Main(string[] args)
5     {
6         Console.WriteLine("Hello, World!");
7     }
8 }
9
```

Platné Main metódy

- V programe je **povolená iba jedna** statická metóda **Main** (pozor: píše sa s veľkým prvým písmenom M) – ak ich máte v projekte viac, je nutné nastaviť, ktorá sa má spúšťať
- V „top-level statements“ si **kompilátor sám zvolí konkrétnu verziu Main metódy podľa zdrojového kódu** (ak použijete v kóde return, zvolí verziu s návratovým typom int, v prípade asynchrónneho kódu sa použije Task/Task<int>)
- Platné deklarácie metódy Main (nie je potrebný ani modifikátor public):

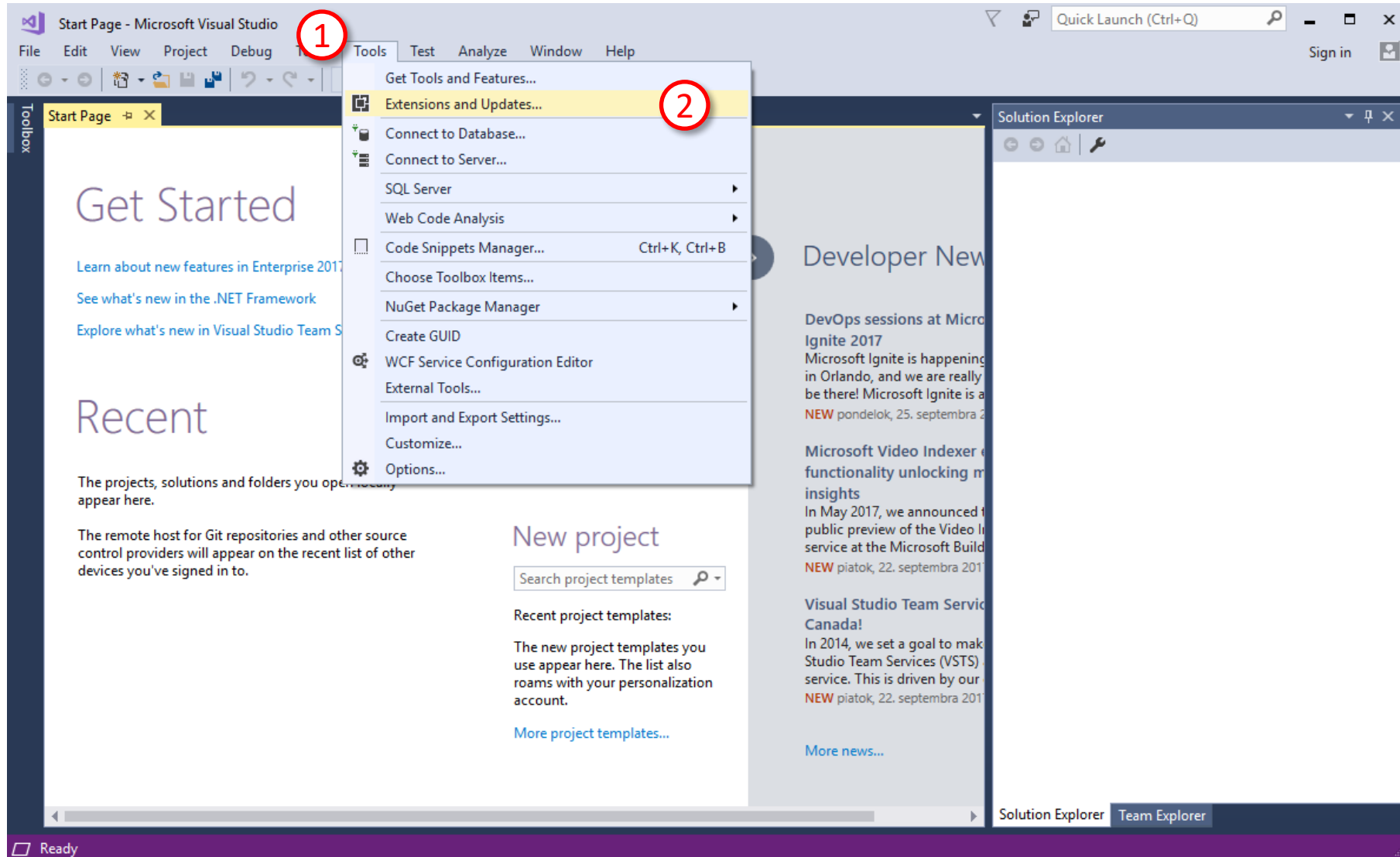
```
public static void Main() { }  
public static int Main() { }  
public static void Main(string[] args) { }  
public static int Main(string[] args) { }  
public static async Task Main() { }  
public static async Task<int> Main() { }  
public static async Task Main(string[] args) { }  
public static async Task<int> Main(string[] args) { }
```

ReSharper (voliteľne) Inštalácia a aktivácia

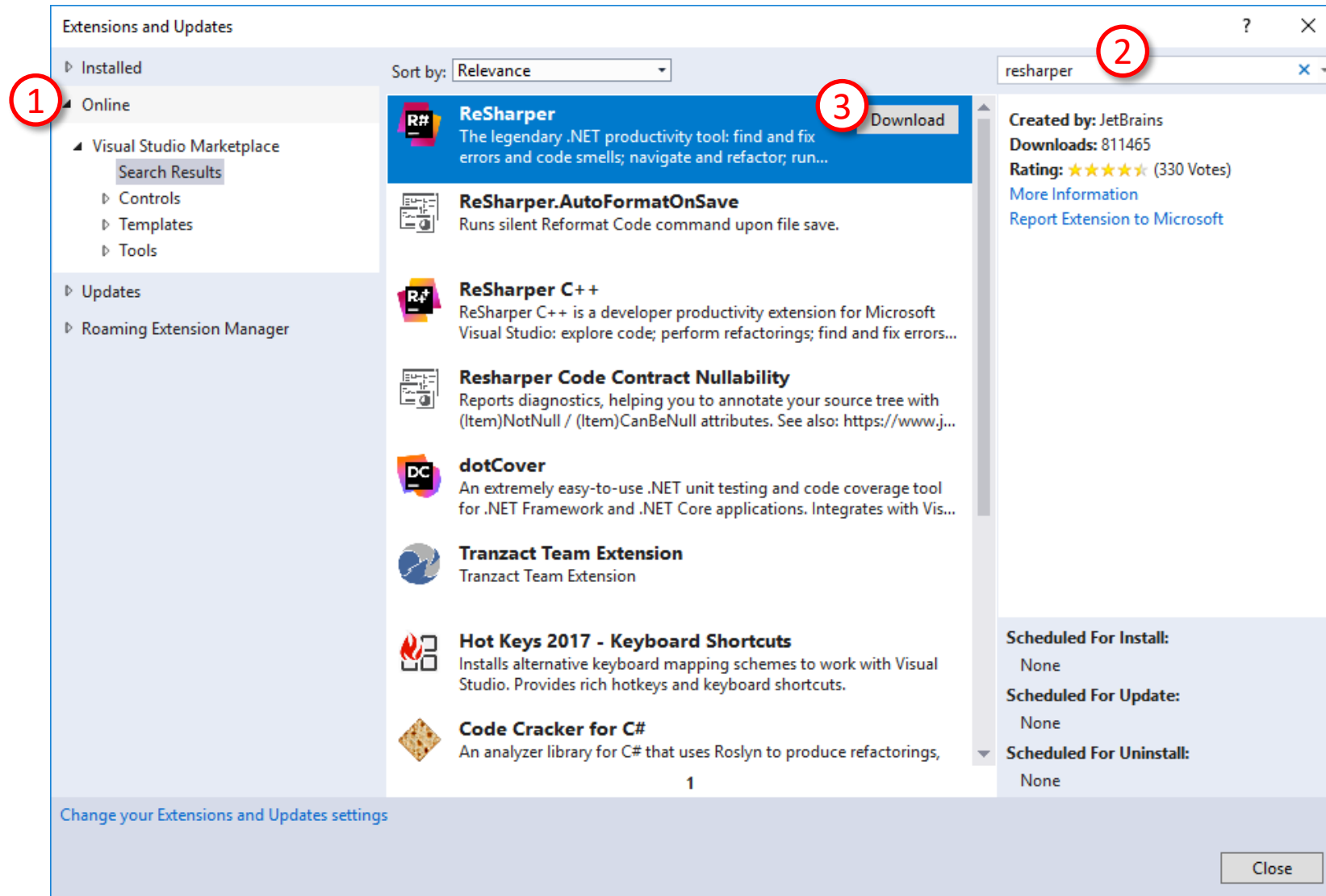
ReSharper – inštalácia

- JetBrains ReSharper je rozšírenie do Visual Studio:
 - analyzuje kvalitu zdrojového kódu, pomáha pri učení jazyka C# a .NET,
 - eliminuje chyby – upozorňuje na ne a ponúka riešenia na opravu,
 - bezpečne refaktoruje zdrojový kód na pokročilej úrovni,
 - umožňuje rýchlu navigáciu a vyhľadávanie,
 - ...
- Možnosť stiahnutia JetBrains ReSharpera:
 - priamo zo stránok spoločnosti (<https://www.jetbrains.com/resharper/>) alebo
 - vyhľadaním z Visual Studio (menu **Tools – Extensions and Updates...** – vľavo kliknúť na **Online** – vpravo hore vyhľadať ReSharper – stiahnuť tlačidlom **Download**)

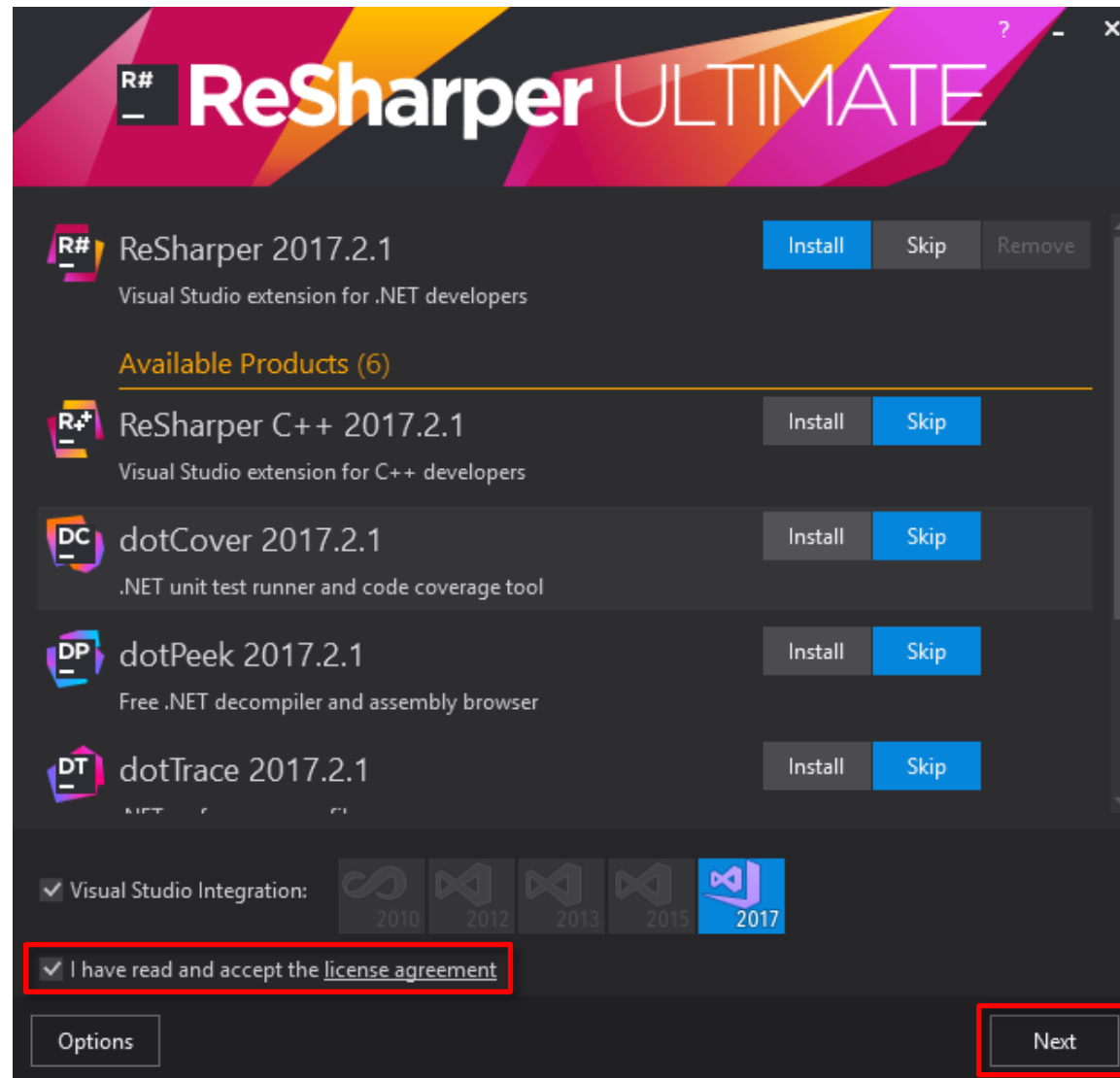
ReSharper – stiahnutie pomocou Visual Studio



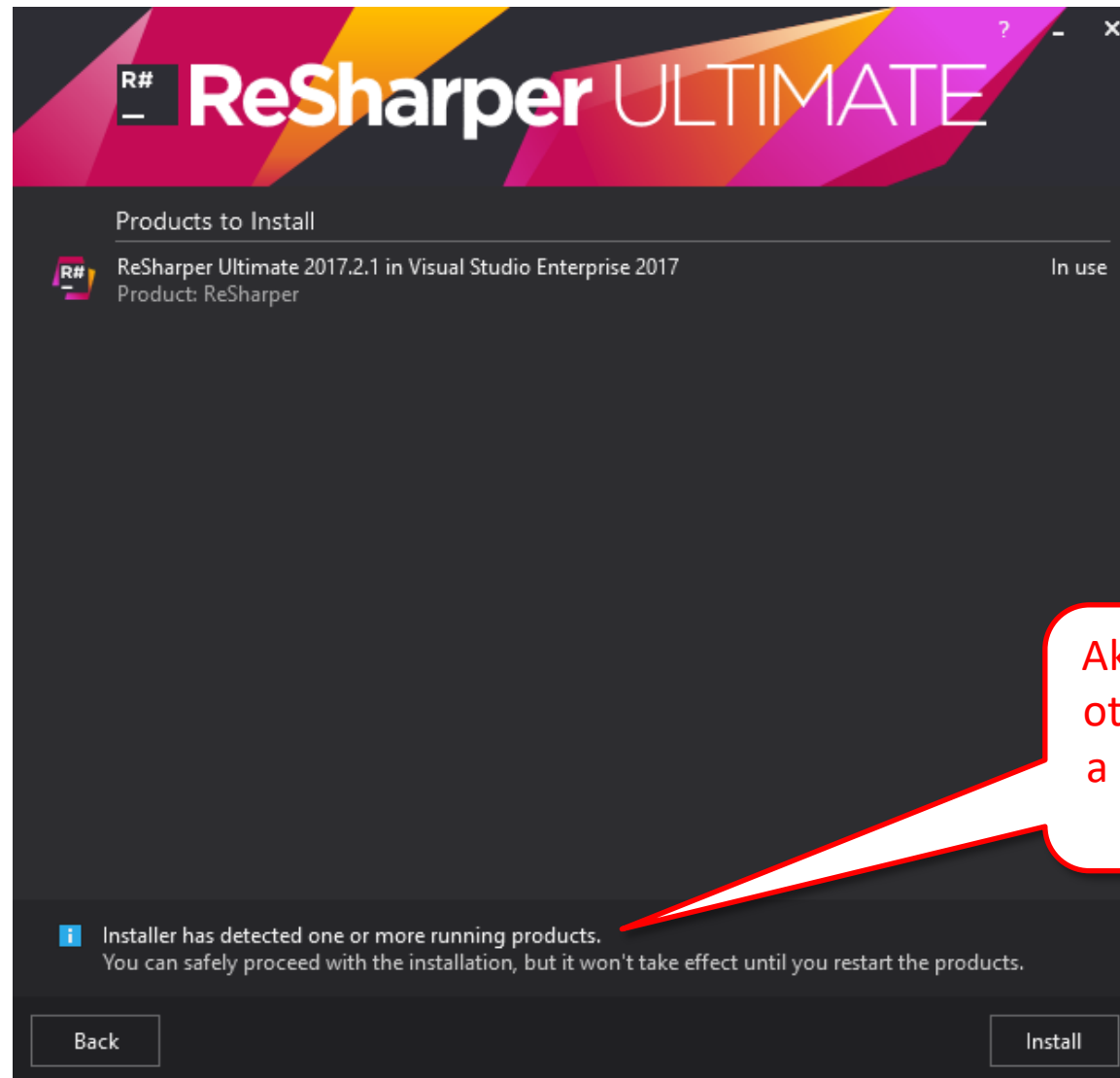
ReSharper – stiahnutie pomocou Visual Studio



ReSharper – inštalácia

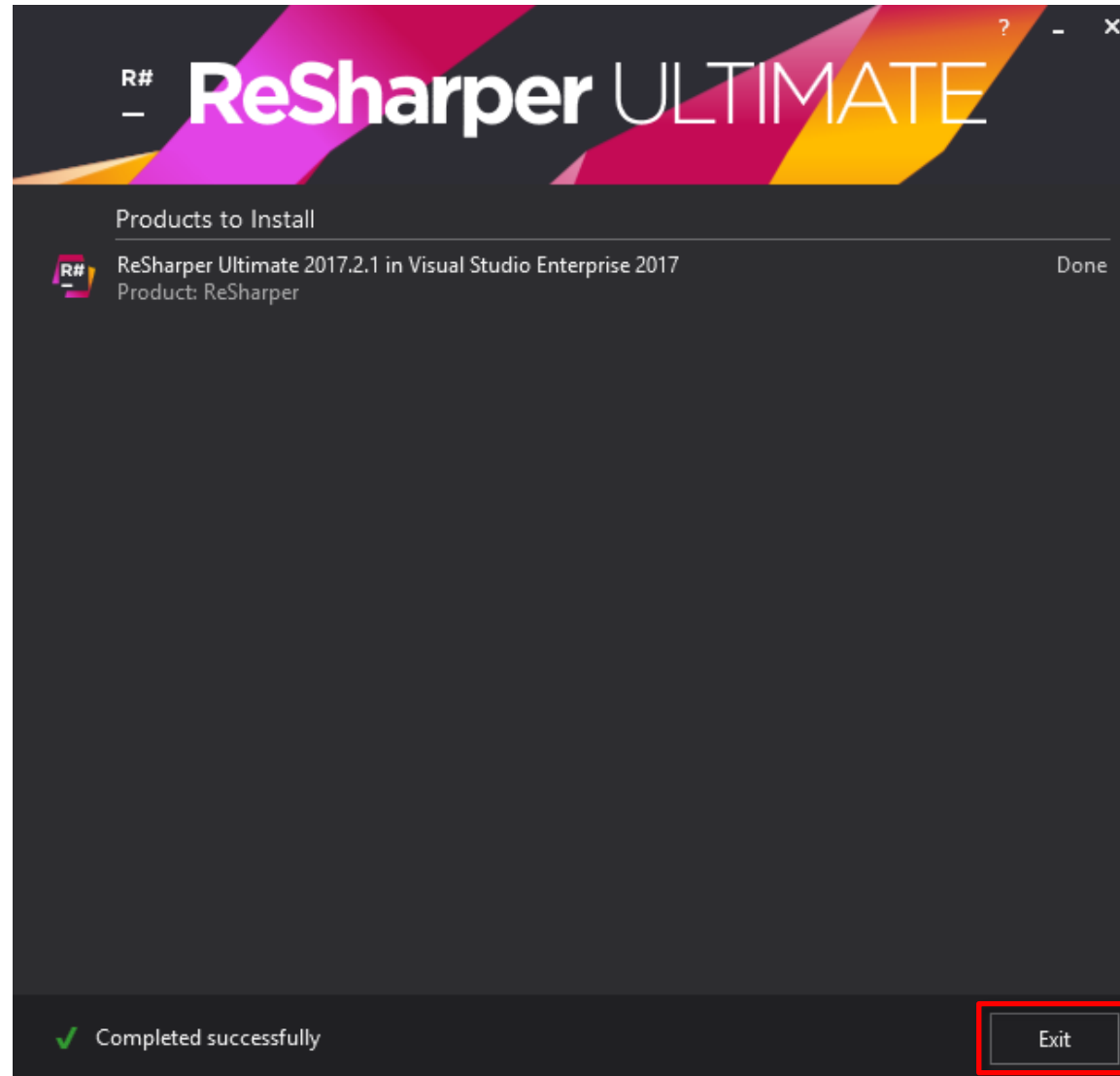


ReSharper – inštalácia

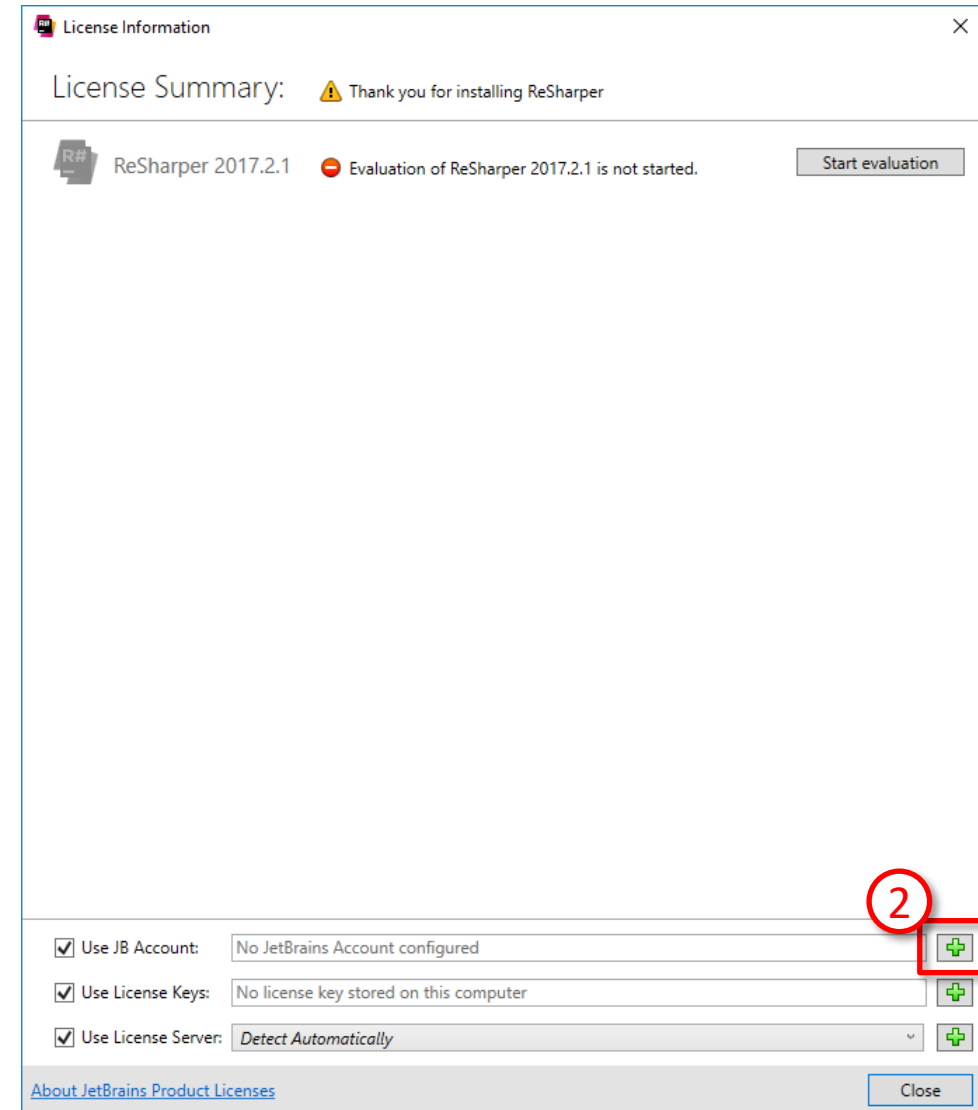
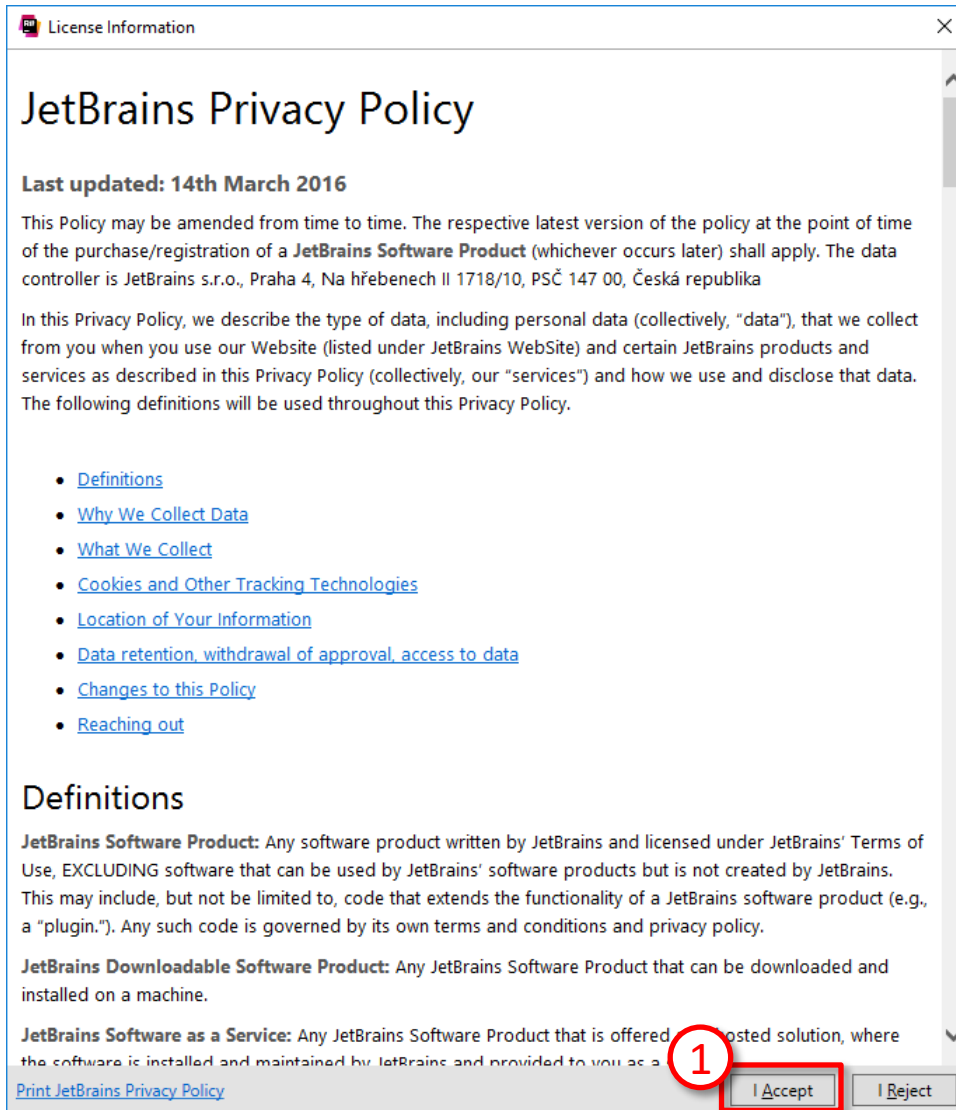


Ak máte Visual Studio otvorené, zatvorte ho a po nainštalovaní ho opäť spustite

ReSharper – inštalácia



ReSharper – aktivácia (po spustení Visual Studio)



ReSharper – aktivácia (po spustení Visual Studio)

License Information

Manage JetBrains Account

Account: stefantoth

Password: *****

Log In

Available licenses:

Not logged in

About JetBrains Account

OK Cancel

Zadajte meno a heslo JetBrains (JB) konta.

Ak JB konto ešte nemáte, zaregistrujte si ho
najskôr pomocou tejto stránky:

<https://www.jetbrains.com/shop/eform/students>

Na overenie statusu študenta je nutné použiť buď
univerzitnú e-mailovú adresu alebo ISIC.

License Information

Manage JetBrains Account

Account: stefantoth

Password: *****

Log Out

✓ Logged in

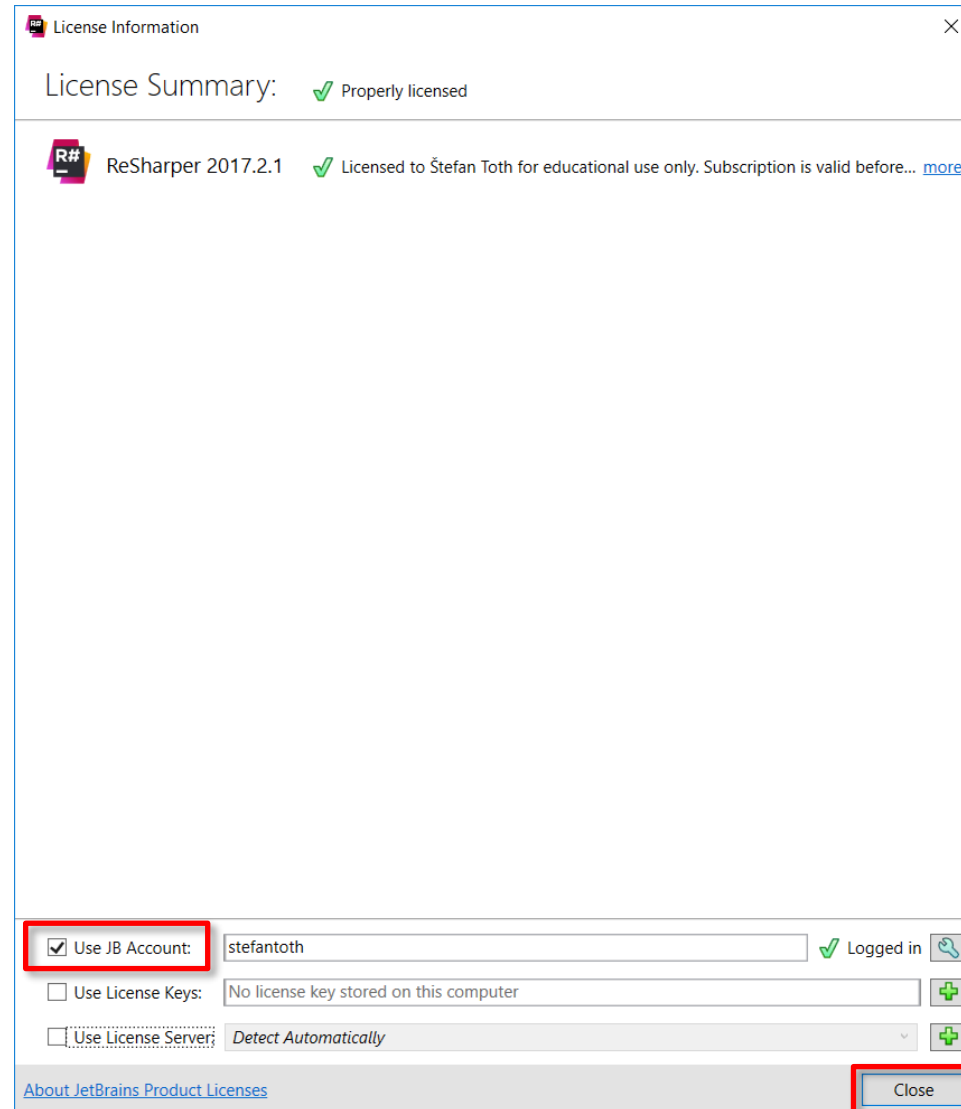
Available licenses:

☒ v2017 962c7e3c08 Licensed to Štefan Toth for educational use only. Subscription is valid before 14. 3. 2018

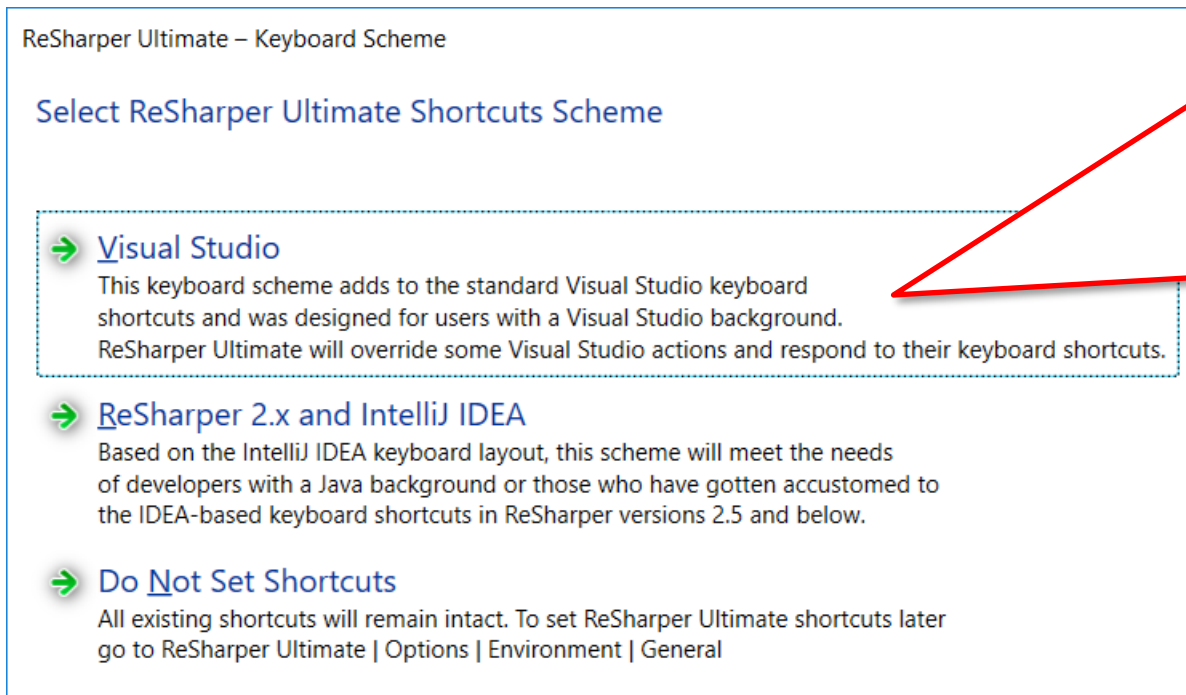
About JetBrains Account

OK Cancel

ReSharper – aktivácia (po spustení Visual Studio)



ReSharper – klávesové skratky



Umožňuje vybrať klávesové skratky používané pre ReSharper.

Ak nie ste zvyknutý používať skratky v IntelliJ IDEA, odporúčam použiť prvú možnosť – **Visual Studio**. Inak môžete zvoliť druhú možnosť.

Zoznam klávesových skratiek pre schému Visual Studio:

https://www.jetbrains.com/resharper/docs/ReSharper_DefaultKeymap_VSscheme.pdf

Zoznam klávesových skratiek pre schému ReSharper 2.x/IntelliJ IDEA

https://www.jetbrains.com/resharper/docs/ReSharper_DefaultKeymap_IDEAscheme.pdf

ReSharper – najpoužívannejšie klávesové skratky

- Ctr + Space
- Alt + Ins
- Ctrl + T
- Shift + F12
- Alt + Enter
- Ctrl + Shift + R

Zdroj:

https://www.jetbrains.com/resharper/docs/ReSharper_DefaultKeymap_VSscheme.pdf



Create

Symbol code completion	Ctrl + Space
Smart code completion	Ctrl+Alt+Space
Import symbol completion	Shift + Alt + Space
Complete statement	Ctrl+Shift+Enter
Parameter info	Ctrl+Shift+Space
Go to previous signature (in parameter info)	Ctrl+Shift+Alt+Space
Quick documentation	Ctrl+Shift+F1
Move code up/down/left/right	Ctrl+Shift+Alt+↑/↓/←/→
Extend selection	Ctrl+Alt+→
Shrink selection	Ctrl+Alt+←
Duplicate a line or selection	Ctrl+D
Select containing declaration	Ctrl+Shift+[
Comment/uncomment line	Ctrl+Alt+/
Comment/uncomment block	Ctrl+Shift+/
Generate code (constructors, properties, implementing/overriding members, etc)	Alt+Ins
Insert live template	Ctrl+E, L
Surround with template	Ctrl+E, U
Create file from template	Alt+Ins in Solution Explorer or Ctrl+Alt+Ins
Go to last edit location	Ctrl+Shift+Backspace
View recent files	Ctrl+,
View recent edits	Ctrl+Shift+,
Go to related files	Ctrl+Alt+F7
View bookmarks	Ctrl+`
Go to bookmark	Ctrl+[numeric key]
Set/remove bookmark	Ctrl+Shift+[numeric key]

www.jetbrains.com/resharper

DEFAULT KEYMAP

Explore

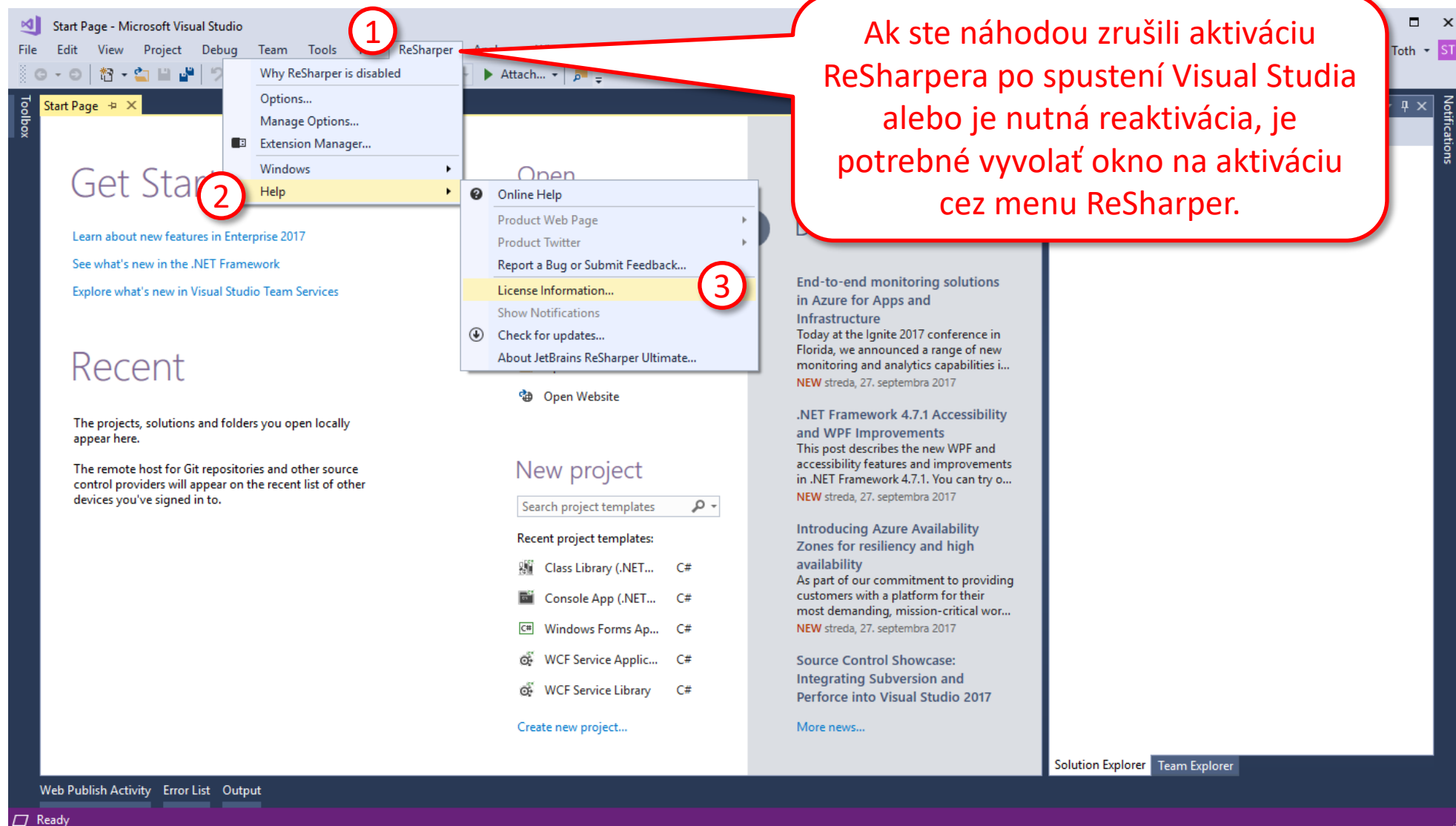
Go to everything	Ctrl + T
Go to type	Ctrl + T (second hit)
Go to symbol	Shift + Alt + T
Go to word	Ctrl + T + T + T
Go to file	Ctrl + Shift + T
Go to file member	Alt + \
Find usages	Shift + F12
Find usages (advanced)	Ctrl + Shift + Alt + F12
Find Results window	Ctrl + Alt + F12
Highlight usages in file	Shift + Alt + F11
Go to previous usage	Ctrl + Alt + PgUp
Go to next usage	Ctrl + Alt + PgDn
Analyze References window	Ctrl + Alt + Y
Navigate to	Alt + `
Go to declaration	F12
Go to type of symbol	Ctrl + Shift + F11
Go to implementation	Ctrl + F12
Go to base symbols	Alt + Home
Go to derived symbols	Alt + End
Go to usage	Shift + Alt + F12
File structure	Ctrl + Alt + F
Go to next member/tag	Alt + I
Go to previous member/tag	Alt + J
Go to containing declaration	Ctrl + [
To-do items	Ctrl + Alt + D
Explore stack trace	Ctrl + E, T
Locate in Solution Explorer	Shift + Alt + L
Structural navigation	Tab/Shift + Tab

Visual Studio scheme

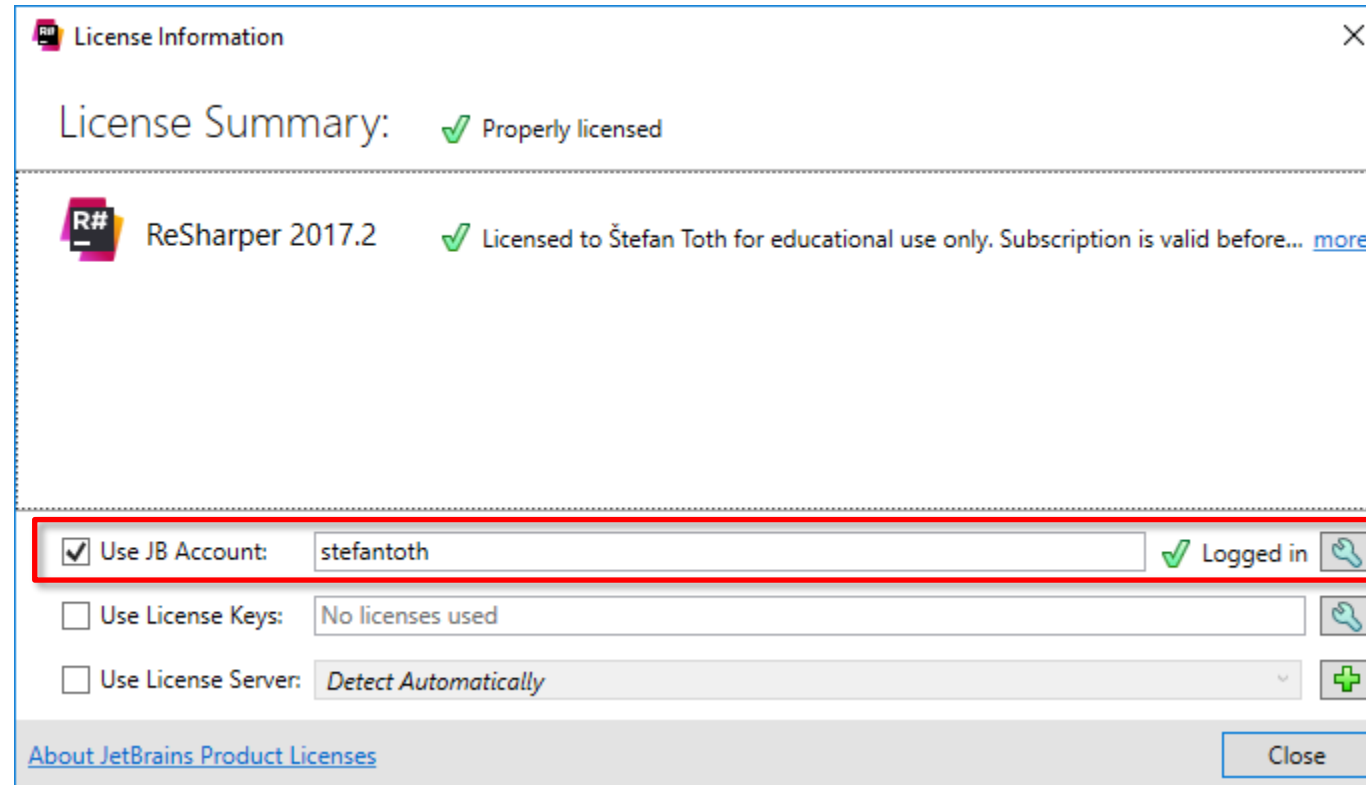
Improve

Show available quick-fixes and context actions	Alt + Enter
Inspect this	Ctrl + Shift + Alt + A
Inspection Results window	Ctrl + Alt + V
View type hierarchy	Ctrl + E, H
Go to next highlight (error, warning or suggestion)	Alt + PgDn
Go to previous highlight (error, warning or suggestion)	Alt + PgUp
Go to next error	Shift + Alt + PgDn
Go to previous error	Shift + Alt + PgUp
Refactor this	Ctrl + Shift + R
Rename	Ctrl + R, R
Move type or static member	Ctrl + R, O
Safe delete	Ctrl + R, D or Alt + Del
Extract method	Ctrl + R, M
Introduce variable	Ctrl + R, V
Introduce field	Ctrl + R, F
Introduce parameter	Ctrl + R, P
Inline	Ctrl + R, I
Change signature	Ctrl + R, S
Code cleanup	Ctrl + E, C
Apply code style	Ctrl + Alt + S
Reformat code	Ctrl + Alt + Enter
Run unit tests	Ctrl + U, R
Debug unit tests	Ctrl + U, D
Run all tests from solution	Ctrl + U, L
Run current session	Ctrl + U, Y
Repeat previous run	Ctrl + U, U
Unit Test Explorer	Ctrl + Alt + U
Unit Test Sessions	Ctrl + Alt + T

ReSharper – aktivácia (cez menu Visual Studio)



ReSharper – aktivácia (cez menu Visual Studio)



Visual Studio

Použitie

Trieda Console

- **Console** – statická trieda, ktorá obsahuje metódy pre zobrazenie výstupu a načítanie vstupu na obrazovke a ďalšie metódy a vlastnosti súvisiace s príkazovým riadkom
- Zapisovanie na výstup
 - `Console.Write()`, `Console.WriteLine()`
- Načítanie vstupu
 - `Console.Read()`, `Console.ReadLine()`, `Console.ReadKey()`
- Zmena farby
 - `Console.BackgroundColor`, `Console.ForegroundColor`

Console.WriteLine() – zložené formátovanie

```
public static void WriteLine(string format, params object[] arg);
```

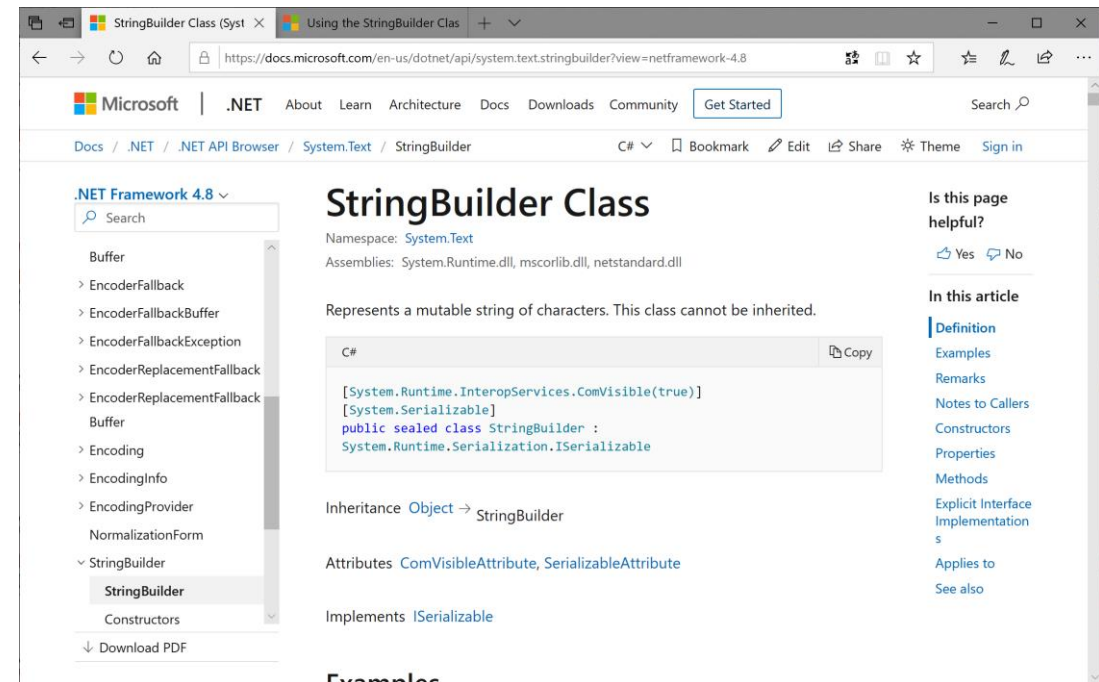
- `format` – zložený formátovací reťazec sa skladá z nula a viac formátovacích položiek v tvare `{index[,alignment][:formatString]}`, ktoré sa odkazujú na ďalšie parametre metódy

- Príklad:

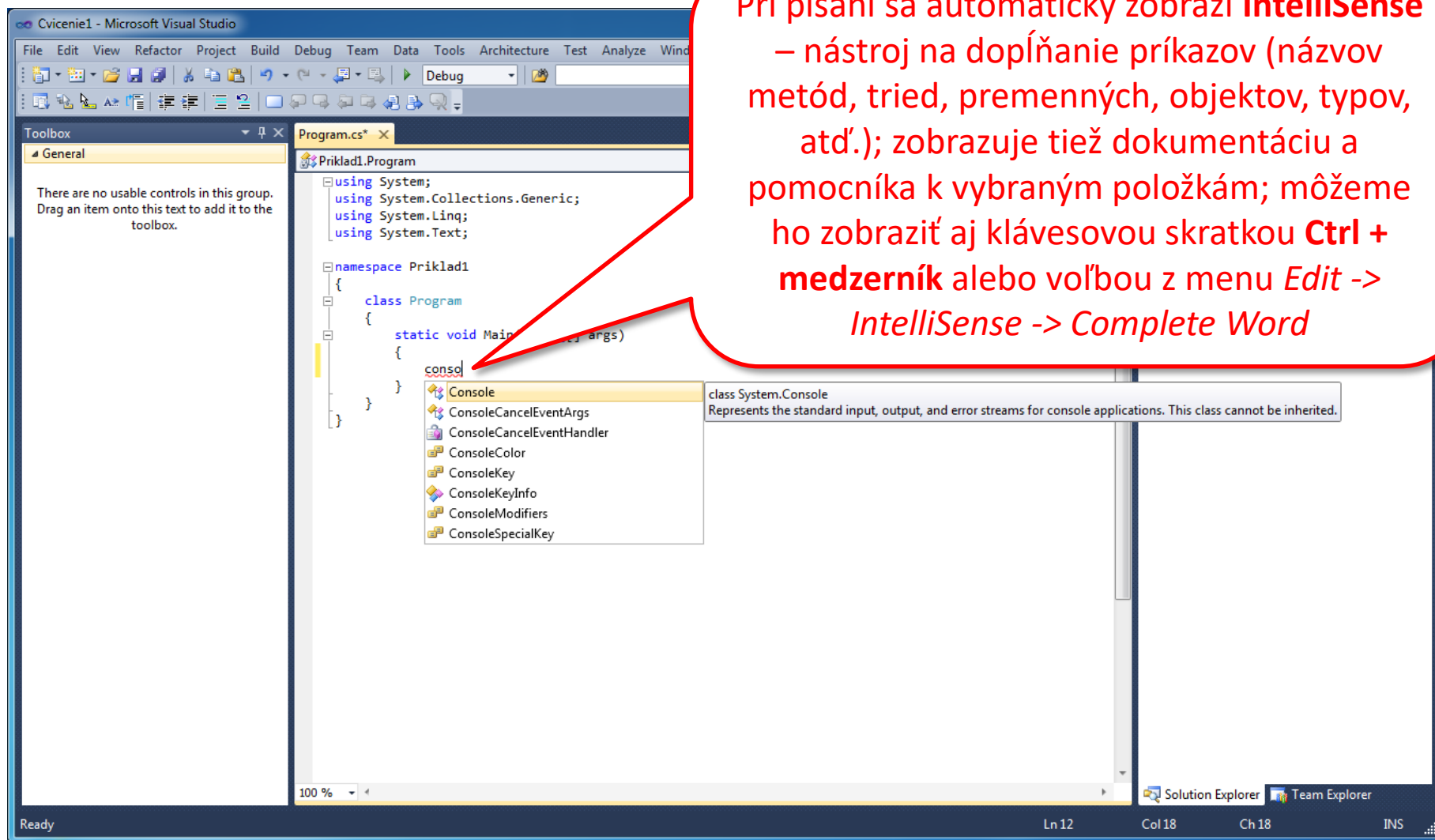
```
// Vypíše "1 pivo stojí 1.20 EUR."
```

```
Console.WriteLine("1 {0} stojí {1:F} EUR.", "pivo", 1.2);
```

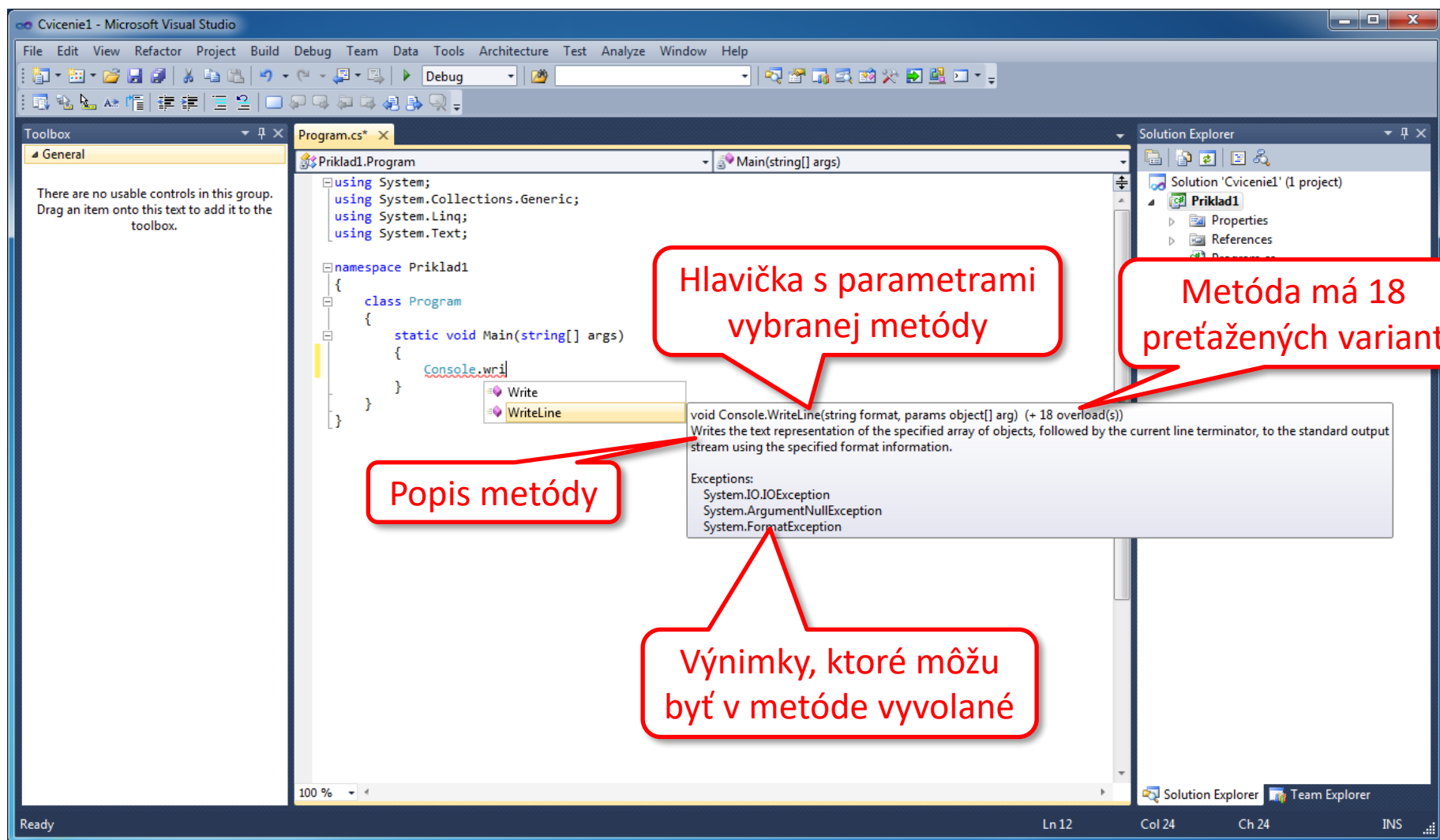
- **string** (**System.String**) – základná trieda
- **System.Text.StringBuilder** – efektívnejší než string (ale nie vždy!), využíva alokovanú kapacitu v pamäti, viac informácií získate na <https://learn.microsoft.com/en-us/>



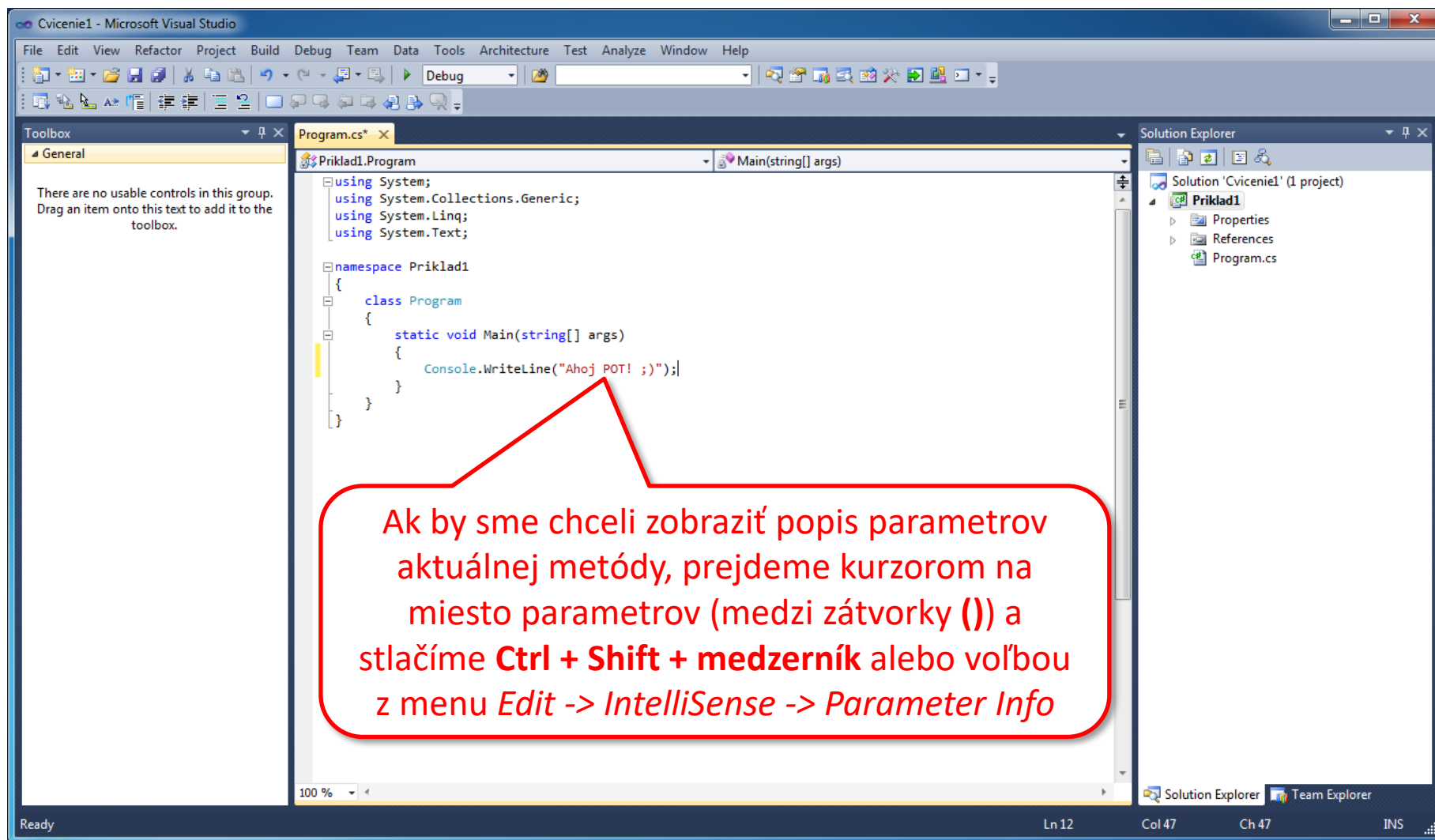
IntelliSense



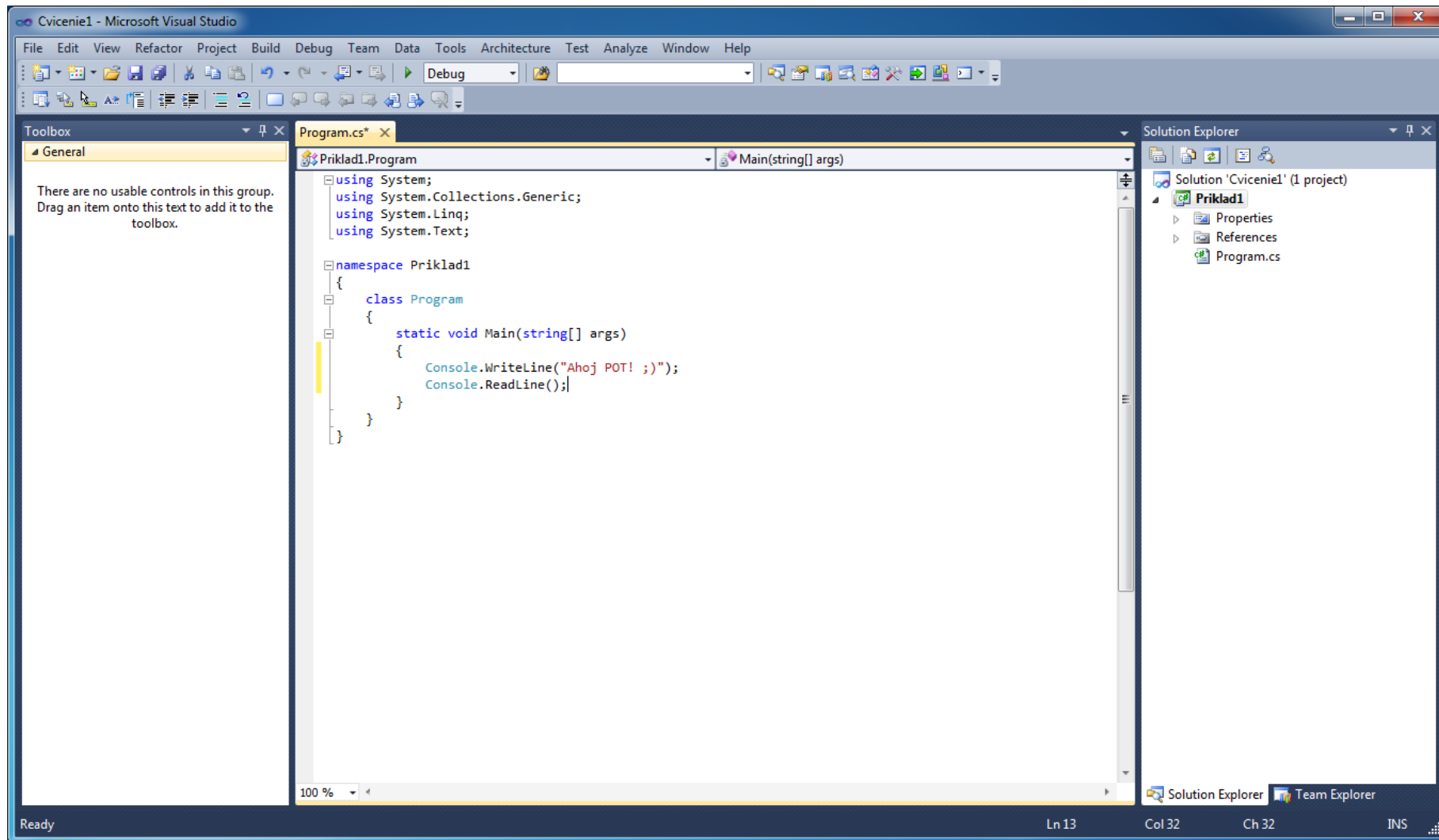
IntelliSense – dokumentácia



Ukážková aplikácia

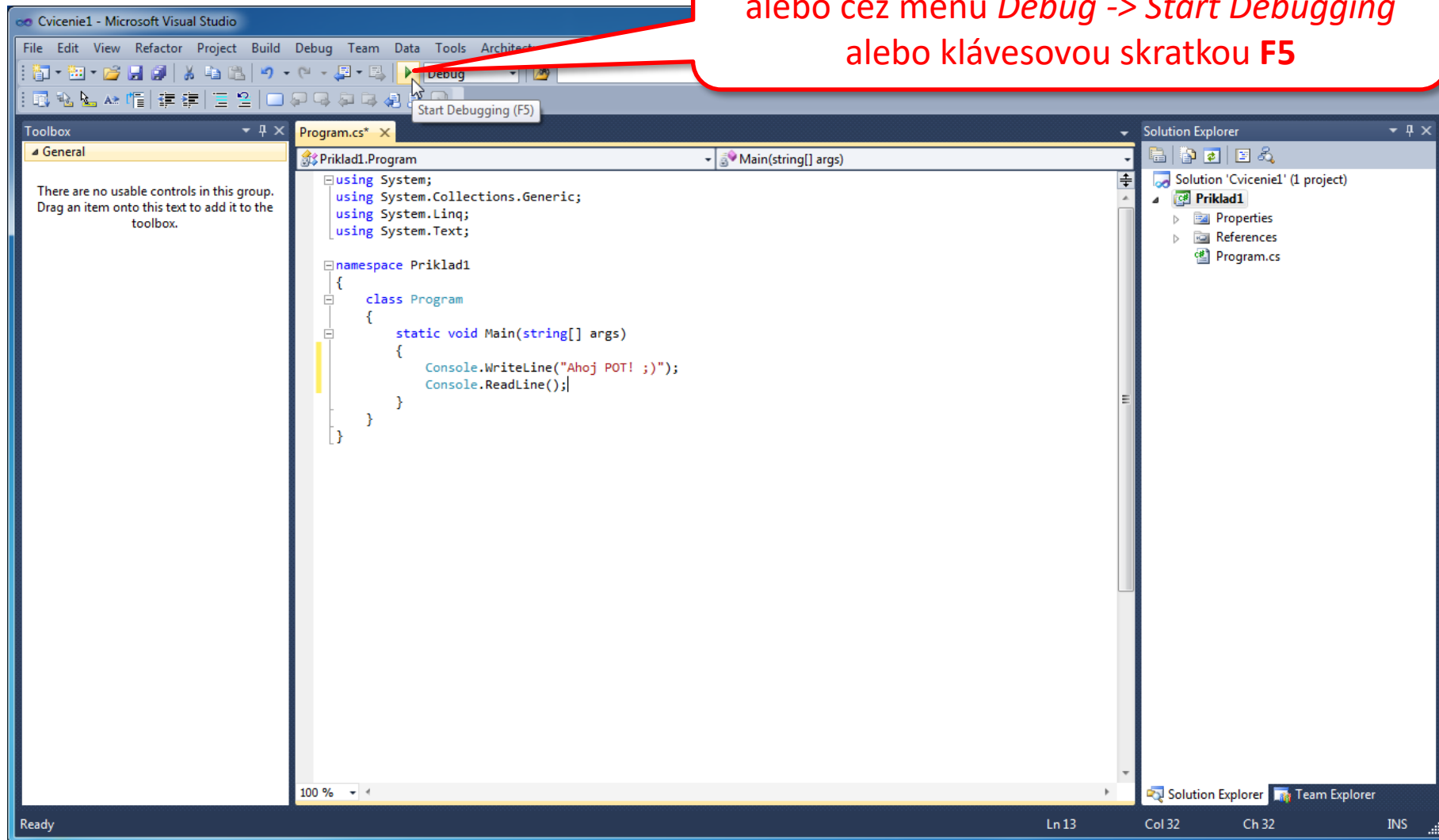


Ukážková aplikácia

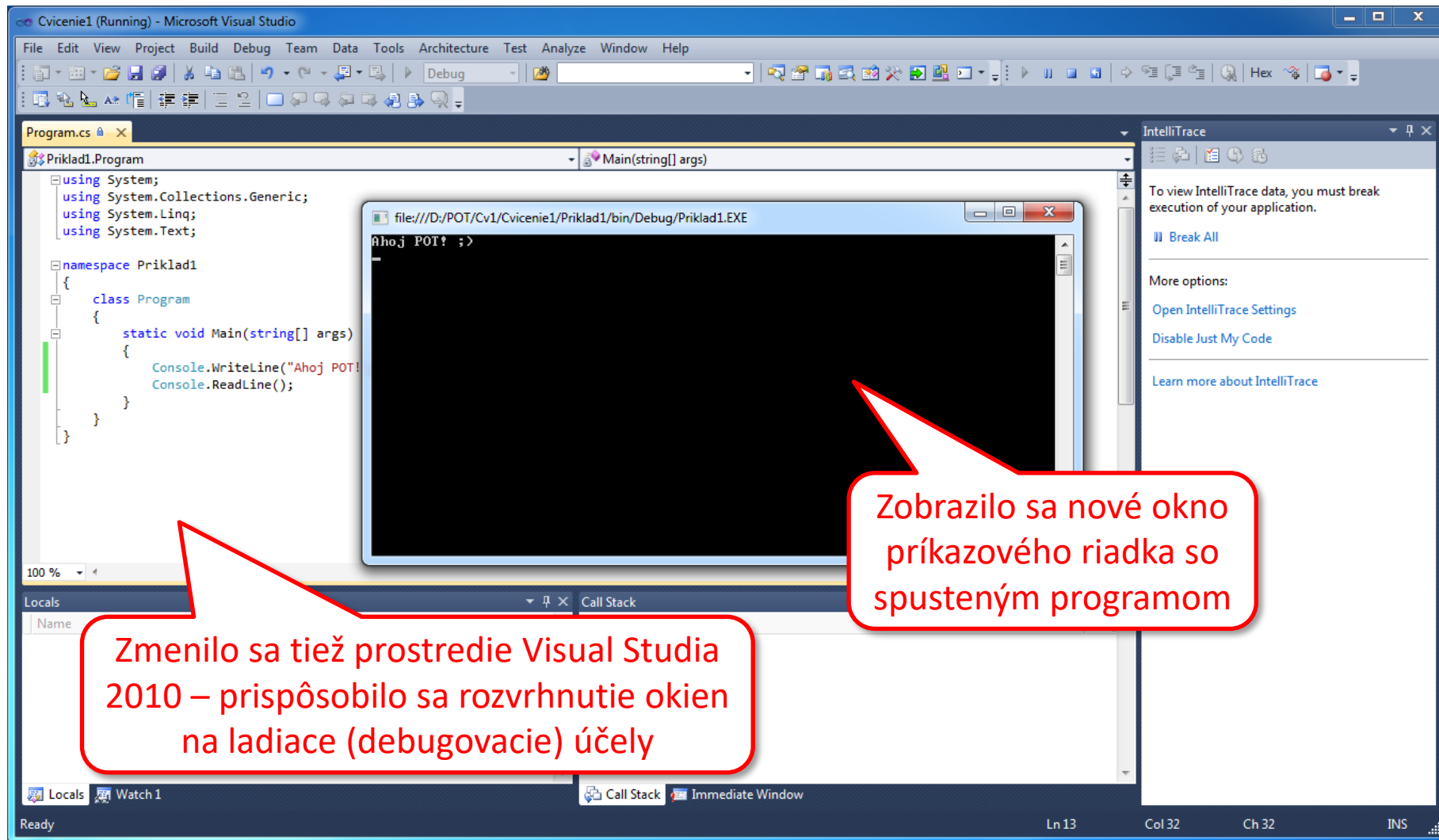


Spustíme aplikáciu

Spustíme aplikáciu tlačidlom *Start Debugging* alebo cez menu *Debug -> Start Debugging* alebo klávesovou skratkou **F5**



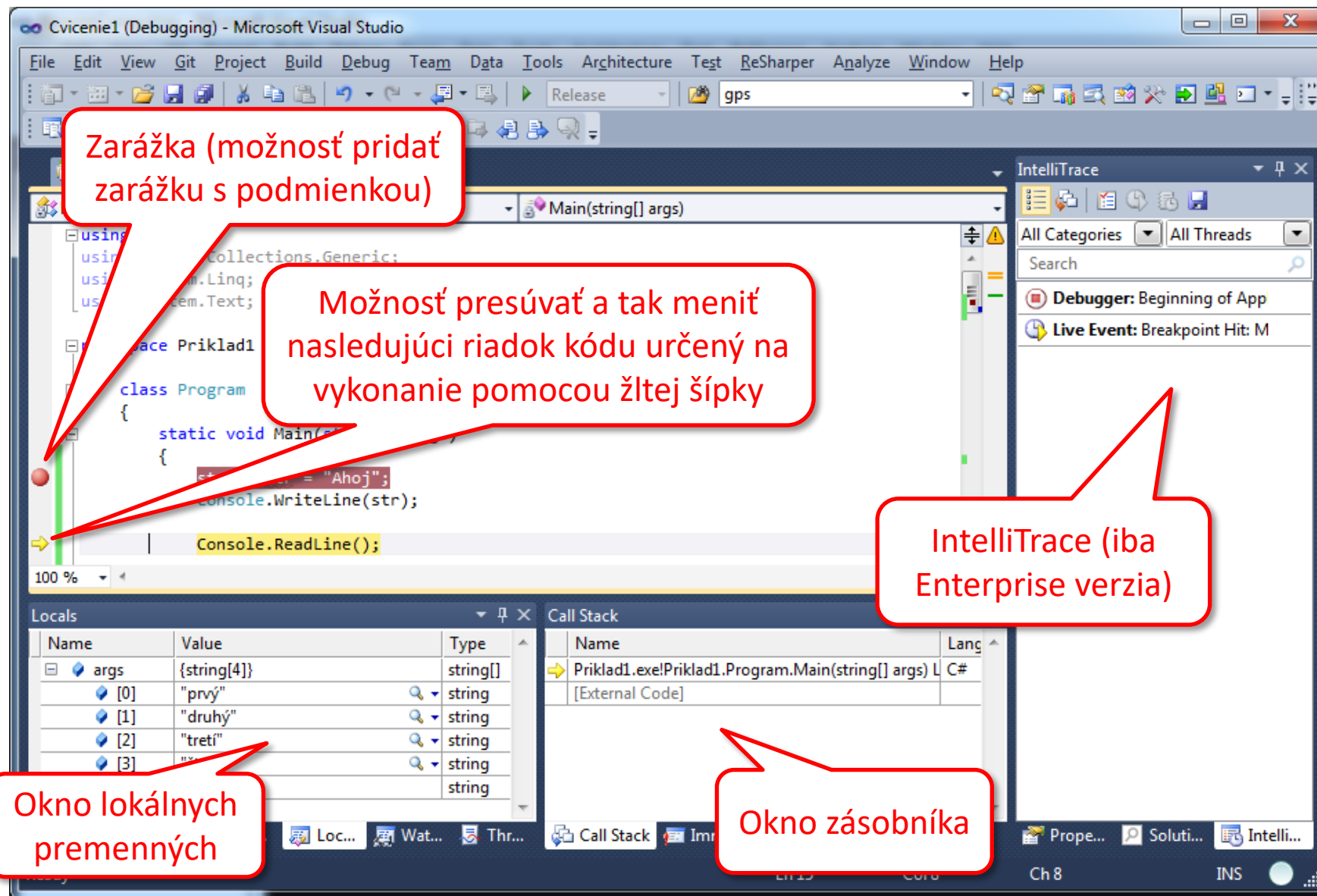
Aplikácia beží...



Ladenie (debugovanie) aplikácie

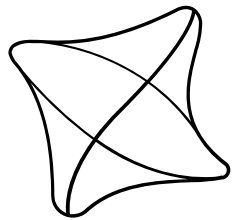
- **Zarážka** (breakpoint) – **F9**, zarážka s podmienkami
- **Inkrementálna kompilácia** – počas debugovania možnosť písať kód za behu aplikácie
- Možnosť manuálne premiestňovať aktuálne vykonávaný riadok kódu (žltá šípka)
- **IntelliTrace** – debugovanie do „minulosti“ (výnimky, stav zásobníka a iné ladiace informácie dostupné aj po ukončení aplikácie)

Ladenie



Upozornenie

- Tieto študijné materiály sú určené výhradne pre študentov predmetu **Pokročilé objektové technológie** na Fakulte riadenia a informatiky Žilinskej univerzity v Žiline
- Reprodukovanie, šírenie (i častí) materiálov bez písomného súhlasu autora nie je dovolené



ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Ing. **Štefan Toth**, PhD.
Katedra softvérových technológií
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
stefan.toth@fri.uniza.sk